基于时空预约闭包的柔性作业车间有界重调度

QI YU, Southeast University, PR China

WEI LI*, Southeast University, PR China

在柔性作业车间调度中，若既用工序依赖图刻画加工先后（同一工件的工序先后、同一机器上的加工先后），又用时间窗预约表记录各车辆占用走廊的时段，扰动发生后划定重调度范围的现成做法仍是受影响工序重调度（AOR）：先找出因扰动而无法按原计划执行的工序，再沿工序依赖图把其后续工序一并纳入重排。走廊阻断或通行变慢这类扰动只作用于走廊，机器与工序均照常可用，AOR 因而找不到任何受影响工序，给出空的重调度范围，判定无需重调度。实际上，原计划中与受扰时段重叠的走廊占用已经失效，原排程不可行；并且同一走廊同一时刻只容一辆车，原计划中随后通过该走廊的车辆须等前车驶离（下称让行关系），前车受阻延误后，后车随之推迟，延误沿让行关系逐车传递。问题不在于扰动未被发现（阻断本身是已知的），而在于 AOR 划界所依据的工序依赖图只含工序及其先后关系，既没有走廊占用，也没有占用之间的让行关系，因而无法表达这类失效及其传播。

本文改为在走廊占用这一层上划定重调度范围，提出时空预约闭包（STRC）方法，该方法输出的占用集合称为预约影响集。其构造是：以原计划中占用时段与受扰时段重叠的走廊占用为起点，沿占用之间的四类依赖关系——同一走廊上的让行、同一车辆的下一条占用、同一工件下一道工序的占用、同一机器上下一道工序的占用——逐步纳入后续占用，直至没有新的占用可纳入（即取传递闭包）。原计划中每条走廊占用由所经走廊（即路径）、占用时段与承运车辆三项确定：影响集内的占用允许重新规划路径与时段（车辆故障等扰动下还允许换车），影响集外的占用三项均保持原计划。四类依赖关系一经确定，一条占用是否属于预约影响集就只取决于它能否从起点沿这些关系到达，结果唯一，不随之后采用哪种改路算法而变。在影响集上的有界修复分两级：先保持原车辆指派，只为集内占用重新规划无冲突路径；若仍得不到可行排程，再依次把同一工件的后续占用、同一车辆的后续占用、直至决策时刻尚未结束的全部占用纳入可改写范围，逐级重试。

实验在 5 个算例上各取 10 个随机种子（随机种子决定扰动的注入与搜索过程），共 50 个算例–随机种子对。走廊阻断下，AOR 按工序依赖图划出的重调度范围在全部 50 对上均为空，但实际上原排程在这些对上均已不可行。改路程序保持不变、只把划界对象由工序依赖图换为预约影响集后，修复后得到可行排程的对数由 0/50 升至 50/50。预约影响集在全部 50 对上满足两条性质：一是对外封闭，即不存在从集内占用指向集外占用的依赖边；二是修复后集外每条占用的走廊、时段与车辆均与原计划一致。若其余条件不变、只把可改写集合由预约影响集换成「决策时刻尚未结束的全部占用」，算法耗时与完工时间几乎不变，但被改写占用的比例由 0.540 升至 0.604：50 对中按预约影响集划界改写更少、持平、更多的对数依次为 31/18/1（Wilcoxon 符号秩检验，双侧 $p = 8.8 \times 10^{-7}$）。另与热启动全局重调度对照：它以原排程为初始解，在给定计算时限内重新搜索机器指派与加工顺序，同样不改写已结束的占用。有界修复的耗时比它低两到三个数量级，但完工时间 $C_{\max}$（全部工件的完工时刻）在所测各计算时限（0.2–2 秒）下均不优于全局重调度；全局重调度在最短时限下已经占优，其 $C_{\max}$ 又随时限放宽只减不增，故放宽时限也不会出现有界修复反超的情形。

由此可见，可行性的恢复来自划界对象的替换：改路程序未变，划界对象由工序依赖图换为预约影响集后，可行性即得以恢复。改写量的减少则来自预约影响集比「尚未结束的全部占用」更小；但实测中前者平均已占后者的约 92%，只排除了不到一成的占用，因而减少幅度有限，在影响集接近全部尚未结束占用的算例上，二者几乎重合，这一减少也随之消失。在集

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

内改路、集外保持原计划以及修复失败后扩大可改写范围，均沿用已有的局部重调度做法；本文的贡献在于重新确定重调度范围应划定在哪一类对象上，即以走廊占用构成的预约影响集取代工序集合。

CCS Concepts: • **Theory of computation** → **Scheduling algorithms**; • **Applied computing** → *Industry and manufacturing*; • **Computing methodologies** → *Planning and scheduling*.

Additional Key Words and Phrases: 柔性作业车间调度, 有界重调度, AGV 无冲突路由, 时空预约表, 时空预约闭包, 走廊阻断

ACM Reference Format:
Qi Yu and Wei Li. 2026. 基于时空预约闭包的柔性作业车间有界重调度. 1, 1 (November 2026), 32 pages. https://doi.org/XXXXXXX.XXXXXXX

1 引言

1.1 背景与动机

考虑运输的柔性作业车间调度多把工件在机器之间的运输时间取为按位置对索引的常数 [13, 16, 25]，即默认车辆之间不争用走廊。有限车队共享稀疏走廊时，这一假设不再成立 [5]，已有工作因此在开工前用时间窗预约表记录各车辆对走廊的占用时段，联合决定机器指派与无冲突路径 [7, 9, 20, 23]。此时一份排程除机器指派与加工顺序外，还包含一张记录「哪辆车在哪段时间占用哪条走廊」的时间窗预约表。扰动既可能使工序无法按原计划执行，例如机器故障或车辆故障；也可能只使预约表中部分走廊占用失效而不改变任何工序，例如某条走廊在一段时间内被阻断或通行变慢。

上述工作回答的是信息完备、尚未开工时如何排出无冲突排程。车间执行到时刻 t_{now} 并发生扰动之后，问题变为三个：扰动使哪些走廊占用失效；须改写哪些占用，才能使排程在扰动下重新满足走廊排他与禁行约束、从而能够继续执行；以及能否不重新启动种群搜索，在毫秒量级的计算时间内求得这样一份可继续执行的排程。

1.2 工序依赖图无法刻画的缺口

在考虑运输的柔性作业车间动态调度研究中，重调度范围通常沿工序依赖图划定。具体做法是：以因设备故障而无法按原计划执行的工序为起点，把同一工件中位于起点之后的工序，以及同一机器上排在起点工序之后加工的工序纳入范围；也可只从起点沿依赖图向后扩展至多 θ 步，其中 θ 是人为设定的扩展深度上限，用于把重调度范围限制在起点附近、避免沿依赖链扩展到整张排程。随后只对纳入该范围的工序重调度，其余工序保持原计划。已有文献中，带 AGV 运输、并以机器故障或插单为扰动的动态调度研究即按此划界——恢复范围分别取「故障后尚未加工的工序」[26]，与「按扰动影响分为保留、继续、重构三类后的工序与配送任务」[22]（后者为批量流混合流水车间，扰动含车辆故障，但模型中没有预约表）。这一做法即受影响工序重调度（AOR）的思路 [18]。后文以按工序依赖图划界的对照代表这一做法（第 2.1 节）。

这一划界在扰动本身就是工序事件时往往合理——例如机器故障使若干工序无法按原指派执行。然而，当排程用时间窗记录走廊占用时，还存在一类扰动：它使某条走廊在某段时间内不可通行，却不使任何机器或工序失效。典型例子是走廊阻断：走廊在 $[t_a, t_b)$ 时段禁止通行。阻断不修改机器指派或加工顺序，机器与工序照常可用，AOR 的起点——因扰动而无法按原计划执行的工序——因此为空，沿工序依赖图向后扩展也无从开始；划出的重调度范围为空，并据此判为无需重调度。但原计划中与阻断时段重叠的走廊占用已与禁行冲突，原排程不可行。不仅如此，走廊是排他资源，同一时刻只容一辆车，原计划中随后通过同一走廊的车辆须等前车驶离，本文称这种先后为让行关系。受阻车辆等待或改道而延误后，原计划中排在其后通过该走廊的车辆随之推迟，后者又推迟它自己的后车，失效由此沿让行关系在预约表上逐车传播。

本节标题所说的「无法刻画」指：阻断本身是已知的，问题不在检测，而在于 AOR 划界所依据的工序依赖图只以工序为顶点、以工件先后与同机先后为边，按定义不含走廊占用及其让行关系，因而既表达不了占用失效，也表达不了失效在车辆之间的传播。若运输时间仍取为按位置对索引的常数矩阵，走廊不是可争用的资源，预约表也不存在，这类失效在模型中无从表达；因此该缺口只存在于用时间窗记录走廊占用的设定中。

据此，本文区分两类扰动。

- **A 类**（作用于工序依赖图）：机械臂故障、加工时间显著偏差、插单，以及车辆故障。这类扰动使部分工序无法按原计划执行，划定重调度范围时的起点工序集非空；按工序依赖图划界与按预约影响集划界都能给出非空范围，因而不是本文检验的重点。车辆故障归入 A 类的理由见第 3.3 节：它虽不改工序指派，却使该车承运的工序不可执行，经「车辆到所承运工序」的对应关系，工序依赖图上仍有非空的起点工序集。
- **B 类**（只作用于预约表）：走廊阻断、走廊通行变慢，以及局部通行权临时取消等。这类扰动不使任何机器或工序失效，从工序层面看没有任何工序受影响，工序依赖图上的起点工序集因此为空；但与受扰时段重叠的走廊占用非空，以其为起点纳入重调度范围所得的预约影响集也非空。

因此，模型同时用工序依赖图刻画加工先后、用时间窗记录走廊占用时，若重调度范围仍只划在工序依赖图上，B 类扰动便会得到空范围。上述带 AGV 的已有动态调度研究不考虑车辆之间的走廊争用（第 2.4 节），工序依赖图是其唯一可用的划界对象；这一问题只在模型同时含工序依赖图与走廊预约表时出现（第 2.5 节）。图 1 对照这一缺口：(a) 为按工序依赖图划界，图上没有作为起点的工序，因而判为无需重调度；(b) 为按本文提出的时空预约闭包方法（第 4 节）在预约表上划界：与阻断时段重叠的占用成为起点，再沿让行关系与接续关系（同一车辆、同一工件、同一机器上的下一条占用）逐步纳入后续占用，形成预约影响集。

1.3 本文主张

本文有三点主张。

第一，当柔性作业车间用时间窗预约保证车辆无冲突通行时，扰动后的重调度范围应定义在由走廊占用及其让行关系构成的预约依赖图上，而不是只定义在工序依赖图上。

第二，划界方法是时空预约闭包（STRC, Spatiotemporal Reservation Closure），其结果集合称为预约影响集：以与受扰时段重叠、因而失效的走廊占用为起点，沿让行、同一车辆的后续占用、同一工件下一道工序的占用、同一机器上下一道工序的占用，把因此直接或间接失效、须重新安排路径或时段的走廊占用，全部纳入重调度范围。这些依赖关系一经给定，集内占用没有指向集外占用的依赖边，「改写哪些占用」因而对应一个可以检验的集合，而不是人为设定的搜索半径。

第三，得到影响集之后，在集内为原车辆改路、集外保持原占用；若仍得不到可行排程，再按同一工件、同一车辆、直至全部未完成占用的顺序，扩大允许改写的范围。A 类故障上另允许换车与改派，仍不改加工顺序。这些步骤沿用既有做法，用于落实影响集并恢复可行排程。

上述三点共用一个立场：**预约影响集只回答「哪些占用已经失效」，不回答「排程应当怎样改」。**本文只用占用之间的依赖关系判断哪些占用已经失效：从起点出发，沿依赖关系能到达的占用属于影响集，到达不了的不属于。这些关系不用于引导搜索或更换指派与路径，后者属于改进排程的决策问题，不在本文范围内。

纳入哪些依赖边是建模选择；边集选定后，影响集对外封闭（集内占用没有指向集外的依赖边）即为定义的推论（命题 2）。影响集不一定是恢复可行所需的最小占用集合，因为最小集合依赖于允许的修复动作，而影响集的定义不绑定于任何一种修复方法（注 2）。

相对以原排程为起点、在限定时间内重新搜索的全局重调度，预约影响集回答的是另一类问题：在必须很快给出可执行方案时，能否恢复可行，并把改写限制在影响集以内。修复时只要先把影响集以外的占用原样写回新的预约表，且最终排程通过冲突校验，影响集以外每条走廊占用的走廊、时段与车辆就与原计划一致（命

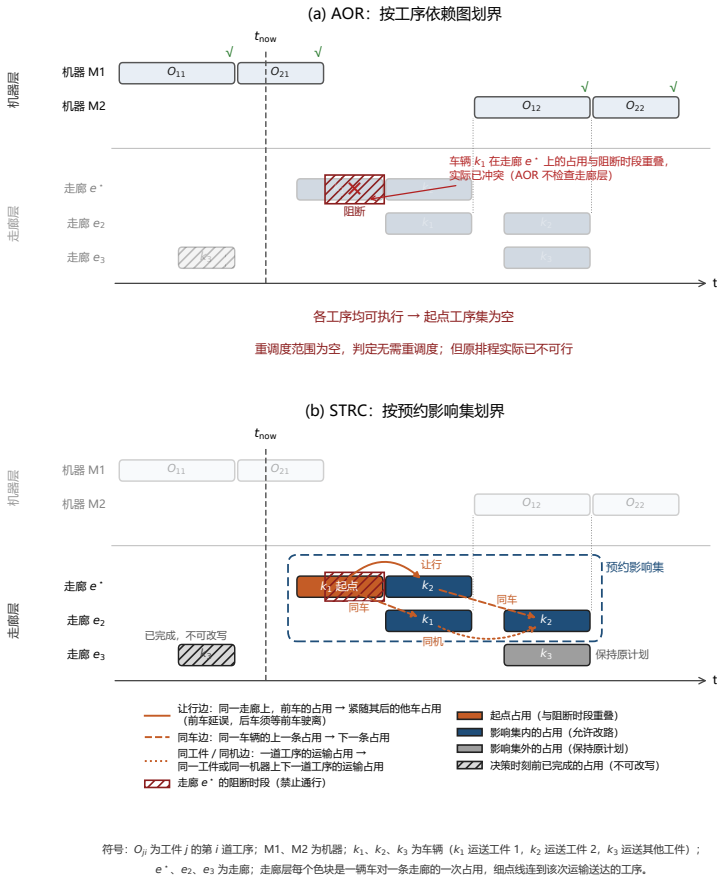


Fig. 1. 动机示意：同一份排程、同一次走廊阻断，两种划界方法检查的层不同。每幅上层为机器层（工序甘特），下层为走廊层（预约表），斜纹框为走廊 e^* 上的阻断时段，竖虚线为决策时刻 t_{now}。(a) AOR 只检查机器层：各工序均可执行，起点工序集为空，判定无需重调度；但走廊层上车辆 k_1 的占用已与阻断冲突，原排程实际不可行。(b) 本文方法检查走廊层：与阻断重叠的 k_1 占用成为起点，沿让行边（$k_1 \to k_2$，同走廊）、同车边与同机边纳入后续占用，形成预约影响集（虚线框）；集外占用保持原计划，t_{now} 前已完成的占用不可改写。

题 3）。按工序依赖图划界时，走廊阻断因找不到受影响工序而不会触发局部重调度，原排程保持不可行；若改为触发全局重调度，则可以恢复可行，但需要秒级的搜索时间：在 8 工件、4 台机器的算例上，本文所测的搜索时限为 0.2–2 秒，最小时限下仅能完成约一代评估；单次解码的耗时随算例规模增长，在 20 工件、10 台机器、10 辆车时，仅评估一代种群就需约 1.4 秒（第 6.11 节）。与全局重调度（同样不改写决策时刻之前已经结束的占用）相比，有界修复的改动比例处于同一量级，算法耗时短得多（毫秒量级对秒量级），但在所测的 12 个算例–随机种子对上，全局重调度在各搜索时限下得到的 $C_{\max}$ 均更小。耗时短与改写少须分开检验（第 6.9 节）：耗时短来自对影响集内各段运输各做一次路径规划、不重新启动种群搜索，而不来自影响集本身；影响集是否减少了改写，无法从全局重调度对照中得出（两者改动比例已经接近），须另设对照：改路程序相同，唯一差别是允许改写的集合取影响集还是尚未结束的全部占用。因此，本文关注的是须立即恢复可行时走廊占用上失效范围的识别及相应的局部恢复。

1.4 贡献

本文的贡献如下，后文分别记为 C1、C2、C3。

C1. 指出工序依赖图无法刻画的一类失效。当排程同时用工序依赖刻画加工先后、又用时间窗记录走廊占用时，现成的重调度范围仍按受影响工序划定。走廊阻断、走廊通行变慢等扰动不改任何工序指派，按该方法划出的范围为空，并据此判为无需重调度；但与受扰时段重叠的走廊占用已经失效，原排程不可行，等待还会沿让行关系传给后续车辆。在全部 50 对走廊阻断实验中均如此（50/50，第 6.2 节）。其根源在于工序依赖图不含占用之间的让行关系。

C2. 把重调度范围定义为预约影响集。以与受扰时段重叠的走廊占用为起点，沿让行、同一车辆的后续占用、同一工件与同一机器上下一道工序的占用，把因此直接或间接失效、须重新安排路径或时段的占用纳入重调度范围；这一划界方法称为时空预约闭包（STRC）。依赖关系一经给定，一条占用是否属于影响集唯一确定，集内占用没有指向集外的依赖边，因而「改写哪些」对应一个可检验的集合，而不是人为设定的搜索半径，也不随改路算法的实现而变。实验上，在四类扰动与两种划界方法的交叉对照中，按工序依赖图划定的范围仅在走廊阻断与通行变慢下为空；按本文方法划定的预约影响集在四类扰动下均非空，且在机械臂故障与车辆故障下包含前者，即改用预约影响集不会漏掉原方法已纳入的占用（第 6.8 节）。划界本身的收益在于减少改动：改路程序相同时，与不划界、把尚未结束的占用全部纳入改写的做法相比，按影响集划界改动的占用更少（50 对中更少、持平、更多的对数依次为 31/18/1，第 6.9 节）。

C3. 给出与该划界相匹配的局部恢复，并分清各项结果各来自何处。在影响集内为原车辆改路、集外保持原占用；仍不可行时再扩大允许改写的范围。修复步骤沿用既有做法，本项贡献在于通过对照区分各项结果的来源。在同一套走廊路网、时间窗路由与无冲突校验下对照，走廊阻断的 50 对实验中，按工序依赖图划界时没有一对得到可继续执行的排程（0/50），按预约影响集划界时全部得到（50/50）。因此，能否得到可继续执行的排程取决于划界对象，而不取决于改路程序的优劣。有界修复的计算耗时为毫秒级，但完工时间 $C_{\max}$ 在所测各计算时限（0.2–2 秒）下均不优于全局重调度，且放宽时限后这一次序不会反转（第 6.7 节）。与上述不划界的做法相比，按影响集划界的耗时与 $C_{\max}$ 几乎相同，说明毫秒级耗时来自单遍改路、不重新启动种群搜索，而不来自影响集（第 6.9 节）。

同一车间、同一预约表上，开工前的一体化排程用预约表构建无冲突方案 [7, 20, 23]；本文在执行时刻用预约表定位失效范围，二者作用于不同阶段。

1.5 篇章结构

第 2 章按四类回顾相关工作：作业车间如何按工序或时间轴划定重调度范围、铁路调度如何表达排他资源上的占用先后、无冲突多车路径如何做局部重规划，以及考虑运输的柔性作业车间把边界划在何处，并据此说明本文与已有工作的区别。第 3 章给出走廊预约设定与执行期恢复问题：车间、走廊预约表与工序依赖图，两类扰动，以及恢复须满足的条件。第 4 章定义时空预约闭包及其结果集合预约影响集，说明其依赖边与对外封闭等性质。第 5 章说明如何在影响集内为原车辆改路、集外保持原占用，以及仍不可行时如何扩大允许改写的范围，并与全局重调度对照。第 6 章报告可行性、算法耗时、占用改动比例与完工时间，并用案例甘特图在工序时间轴上展示统计结果对应的改动形态。第 7 章总结全文，并讨论局限与后续工作。

2 相关工作

动态调度的一个基本问题是划定重调度范围：扰动发生后，哪些工序、走廊占用或车辆的原有安排须重新规划，范围以外的安排则保持原计划不变。对于重调度范围应划在哪类对象上，已有文献给出了不同做法：工序或时

间轴、排他资源上的占用先后、受影响的车辆，以及带运输的柔性作业车间中的工序集合。下面按这四类回顾各自做了什么、尚未覆盖什么，并在表 1 中把它们与本文在问题设定和重调度范围上逐项对比。

2.1 作业车间的局部重调度

对机器故障、插单等工序事件，按受影响工序划定重调度范围，在带 AGV 的动态调度中仍被采用。Zhang 等 [26] 处理机器故障与插单，恢复范围取故障后尚未加工的工序；Tang 等 [22] 按扰动影响把工序与配送任务分为保留、继续、重构，并把完全重调度、部分重调度与右移并列，指出右移保住稳定性、让出优化空间。这一做法的代表是受影响工序重调度（AOR）及其改进 MAOR [18]：只重排被扰动直接或间接波及的工序。沿时间轴划界与失败后逐步扩大重调度范围同样已有文献研究过。match-up 方法从扰动时刻向后寻找能够回到原排程的时间点，只重排这段时间内的工序：Qiao 等 [15] 将其用于半导体制造系统，并与右移重调度作了比较；Salido 等 [17] 在机器可变速的作业车间中先用 match-up 确定重调度区间，再在区间内降低能耗。增量扩大时间窗的 Local Rescheduling [6] 则在修复失败后逐步放宽范围。右移则不判定谁受影响，把未完成部分整体后推。滚动时域方法把长时域问题拆成相互重叠的短时域子问题依次求解 [8]，分解方法则按优先约束结构把工件集拆成依次排序的子块 [11]；综述见 [14, 24]。

可见，先划定重调度范围、再只在范围内局部重调度，是动态调度中较为通用的做法；但上述工作都以受扰动的工序为划界起点。一旦扰动只使走廊不可通行而不改指派，就没有作为起点的受扰工序，按该方法划出的范围必然为空，原排程却已不可行。

2.2 排他资源上的占用先后

「谁先占用同一排他资源」被建成图，在铁路调度中已有系统研究。Mascis 与 Pacciarelli [12] 的 alternative graph 把析取图推广为：顶点是作业对闭塞区段的占用，固定弧表示路径先后，成对的 alternative arc 表示谁先通过该区段。D'Ariano 等人 [1] 用该模型做列车实时重调度，扰动后重算无冲突时刻表，并尽量接近原计划。alternative arc 刻画的是两列车通过同一闭塞区段的先后次序；本文中两辆车通过同一走廊时的让行关系，也是同一排他资源上的这种占用先后。

在 alternative graph 中，每一处争用对应一对互斥的弧（甲先通过，或乙先通过），排程即从每对中选定一条，使图中不出现正权回路、时刻表可行，并尽量使目标最优；占用先后因此是待优化的决策变量。铁路模型把闭塞区段当作机器，列车的运行与区段占用在同一张图上，区段不可用即直接表现为图上的事件，因此不存在「范围划在工序依赖图上而遗漏走廊阻断」的问题。本文则在计划已排定、先后已确定之后使用这类关系：不再选择先后，只沿既定的让行与接续关系判断扰动后哪些走廊占用须重新规划。

2.3 无冲突多车路径的局部重规划

在多智能体路径规划中，只为受影响者进行重调度、其余视为动态障碍，已是标准做法。Švancara 等人 [21] 在在线 MAPF 中给出 Replan All、Replan Single 与 Replan Affected，并用在线独立性检测确定受影响集合；Malka 等人 [10] 在地图不准确时，同样只为受观测变化影响的车辆重规划。无冲突路径规划把时空占用作为一等约束，但起终点通常由外部给定 [27]。

这类工作只有车辆的起终点，没有工件—工序—机器指派，因而不会出现「重调度范围按工序依赖图划定、走廊阻断因找不到受影响工序而被遗漏」的情形。就受影响集合如何得到而言，MAPF 用独立性检测找出受影响车辆，得到的是检测算法的输出，随实现方式和停止条件而变。本文给出的预约影响集则不同：依赖边一经给定，一条占用是否属于该集，只取决于它能否从起点沿这些边到达，与随后用哪一种改路算法无关（第 4.2 节）。

表 1. 相关工作与本文在问题设定和重调度范围上的对比。✓ 表示具备，✗ 表示不具备，—表示不适用（该类工作不划定局部重调度范围）。「占用时间窗」指模型显式记录各车辆（列车）占用路段（闭塞区段、时空格点）的时段。「划定在」为范围内允许改写的对象；「失效沿哪些途径纳入」以失效所经的共享资源表示划界时追踪的传播途径；「走廊扰动下不漏」指走廊阻断等只作用于走廊的扰动发生时，失效的占用全部落入范围；「与修复算法无关」指范围由给定规则唯一确定，不随修复或检测算法的实现与停止条件而变。

代表工作	问题设定			重调度范围			
	工序与机器指派	占用时间窗	执行期扰动	划定在	失效沿哪些途径纳入	走廊扰动下不漏	与修复算法无关
AOR [18]	✓	✗	工序事件	受影响工序	工件、机器	✗	✓
match-up [15, 17]、LRS [6]	✓	✗	工序事件	时间窗	时间窗逐步放宽	✗	✗
带 AGV 的动态调度 [22, 26]	✓	✗	设备故障、插单	工序与运输	按扰动类型分类	✗	✓
铁路实时重调度 [1, 12]	✗	✓	列车延误	全部列车	—（整体重排）	✓	—
在线 MAPF [10, 21]	✗	✓	新任务、地图变化	受影响车辆	车辆间路径冲突	✓	✗
常数或聚合运输时间 [4, 13, 16]	✓	✗	无（开工前）	—	—	—	—
一体化无冲突排程 [7, 20, 23]	✓	✓	无（开工前）	—	—	—	—
本文	✓	✓	**工序事件与走廊扰动**	**走廊占用（指派与顺序不变）**	**走廊、车辆、工件、机器**	✓	✓

2.4 考虑运输的柔性作业车间调度

考虑运输车辆的柔性作业车间调度已有系统综述 [25]。近期组合优化工作仍普遍把行驶时间取为按位置对索引的常数矩阵，并在此之上发展约束规划与知识引导搜索 [13, 16]；也有工作让运输时间随载重变化，并以聚合的拥堵延误项近似车辆之间的相互影响 [4]。这两种约定都不记录车辆对具体路段的占用，走廊阻断在模型里没有对应对象，无法被检验。

一旦用时间窗记录走廊占用，加工先后与走廊占用同时进入模型。拥堵感知派车可用局部密度等代理量选车，而不改机器指派 [5]。把柔性指派与无冲突运输联合建模的工作相对较少：van Os 与 Basten [23] 形式化了带无冲突运输约束的柔性作业车间调度，并以分解方法在多数基准算例上取得可证最优，少数因限时未能保证最优性；两阶段方案先固定排产再消解车辆冲突 [20]；亦有学习层排产、路径层消冲突、并以有限反馈连接的框架 [7]。但这些工作回答的是开工前如何排出可行且质量好的无冲突排程，并不处理执行期扰动，例如 Sun 等 [20] 假定车辆始终可用、不发生故障。反过来，第 2.1 节中处理执行期扰动的带 AGV 动态调度工作又不建模车辆之间的走廊争用，例如 Zhang 等 [26] 忽略 AGV 之间的碰撞。也就是说，在考虑运输的柔性作业车间调度研究中，走廊预约表与执行期扰动尚未同时出现。二者同时出现时，可直接沿用的只有按受影响工序划界的做法，而它对不改变工序指派的走廊阻断给出空范围（第 6.2 节实测 50/50）。本文研究的正是这一情形。

2.5 本文定位

综上，在考虑 AGV 无冲突路由的柔性作业车间中，排程同时用工序依赖刻画加工先后，又用时间窗记录走廊占用。此时会出现上节所述的情形：走廊阻断等扰动不直接作用于机器或工序，从工序层面看，原有的机器指派与加工顺序都无需改动；但与受扰时段重叠的走廊占用已经失效，原排程不可行，相应运输的延误还会推迟后续工序的开工。如表 1 所示，所列工作中没有一类同时具备工序与机器指派、占用时间窗与执行期扰动三项设定：记录占用时间窗并处理执行期扰动的铁路调度与在线 MAPF 没有工序与机器指派，有工序与机器指派并

处理执行期扰动的工作则不记录占用时间窗，因而不处理走廊扰动。本文在三项设定兼备时，把重调度范围改到走廊占用及其让行、接续关系上，得到由这些关系唯一确定的预约影响集。

表中后两列也说明了本文划定范围的原则。范围的大小以「不漏」为先：失效的占用必须全部纳入，否则修复后的排程仍不可行；在此前提下，沿传播途径不可达的占用不纳入，原计划保持不变，但影响集不追求最小，因为最小集合取决于所用的修复动作（注 2）。传播途径也不是越多越好，而是须与失效实际传播的渠道一致：漏掉一类，就会漏掉经该类资源受波及的占用，AOR 只沿工件与机器两类途径扩展，走廊阻断下因此得到空范围；纳入并不传播失效的关系，则只会无谓扩大范围。本文的四类途径——走廊（让行）、车辆、工件、机器——对应模型中约束走廊占用时刻的四类共享资源。本文范围内的对象是走廊占用：可改其路径与时段（车辆故障下还可换车），工序的机器指派与加工顺序不变；修复失败时，再按既定顺序逐级扩大允许改写的占用集合（第 5 节）。

3 走廊预约设定与执行期恢复问题

3.1 系统设定与记号

本文研究的问题属于带无冲突运输约束的柔性作业车间调度，因此以 van Os 与 Basten [23] 对该问题的形式化为基础建立模型，二者在预约对象上的差异见第 3.1.2 节。本节给出后文各章共用的对象与记号。扰动前的可行排程 $\mathcal{S}^0$ 视为给定输入：后文的划界只依赖 $\mathcal{S}^0$ 本身（机器指派、加工顺序与预约表），不依赖它在 $t = 0$ 由何种算法求得，因此 $\mathcal{S}^0$ 可以来自任意排程方法。

3.1.1 **车间与四组决策.** 车间路由图 $G = (V, E)$ 强连通。每台机器 m 由位于固定节点 $\mathrm{loc}(m)$ 的机械臂执行加工，下文的机械臂故障即指该机器失效；$v_0 \in V$ 为装卸站。n 个工件、机器集合 M、同构单载车队 $K = \{1, \dots, N_A\}$。工件 j 有固定工序链 $O_{j1} \to \cdots \to O_{jn_j}$；工序 O_{ji} 的候选机器集为 $\Omega_{ji} \subseteq M$，在机器 m 上耗时 t^P_{jim}。一份完整排程同时确定四组决策：(i) 每道工序的机器指派 x; (ii) 各机器上的工序加工顺序 π; (iii) 各次搬运的车辆指派 y; (iv) 每次车辆移动在 G 上的无冲突路径。记

$$\mathcal{S} = (x,\ \pi,\ y,\ R),$$

其中预约表 R 是第 (iv) 组的显式记录，定义见下。

运输任务不是事先给定的固定参数，而是由机器指派 x 决定：只有同一工件相邻两道工序被指派到位于不同节点的机器上时，才需要一次搬运；指派一旦改变，搬运的次数与起讫节点也随之改变。每次搬运拆成两个运输段：*空驶段*是车辆从当前位置空载行驶到前一道工序所在机器的节点取件，*载运段*是车辆载着工件从该节点行驶到后一道工序所在机器的节点送达；两段各有自己的任务标签 u，在预约表中是两条独立的记录。装卸时间取为零：载运段在「空驶段抵达取件节点」与「工件完成前一道工序」二者中较晚的时刻出发，送达时刻即后一道工序可以开工、车辆可以接新任务的时刻。后文判定哪些占用允许改写、哪些保持原计划，以及改路，都以单个运输段（空驶段或载运段）为基本单位，而不是以整次搬运或整道工序为单位。

3.1.2 **走廊预约与无冲突.** 每条走廊 $e \in E$ 是排他资源，通行时间 τ_e 已知。走廊按单车道处理：车辆无论朝哪个方向通行，都占用整条走廊，因而任一时刻至多一辆车在 e 上，迎面相遇与同向超车都不会发生。这对应 AGV 车间中常见的单车道窄通道与分区占用控制；双车道走廊可在路网中表示为两条走廊。占用取半开区间：后一辆车可以恰好在前一辆车驶离的时刻进入。车辆只在节点处停靠等待，不在走廊中途停车；节点设容量充裕的停靠位，因而节点上的等待不受限制。一条预约记录为

$$r = (e,\ [t^s, t^e),\ k,\ u) \in R,$$

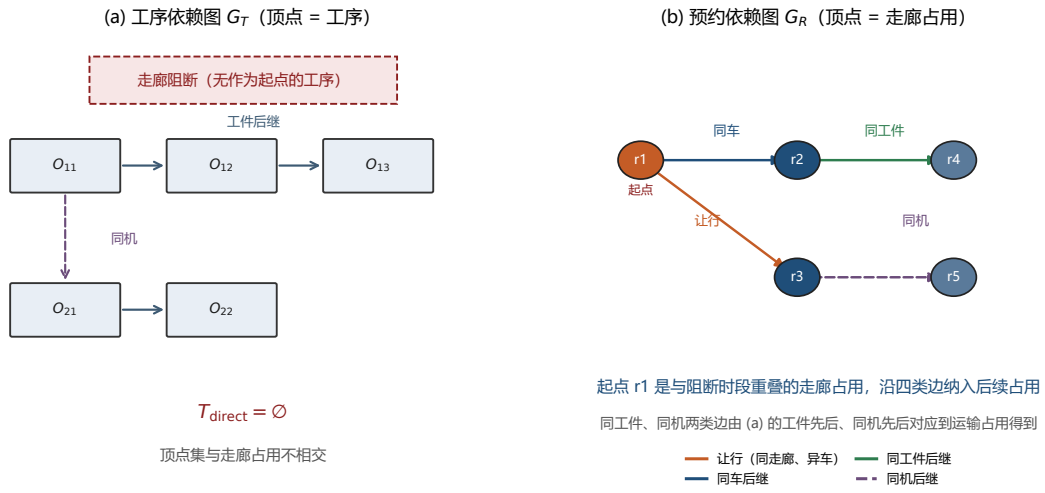


Fig. 2. **两张图的顶点集不相交（构造示意）。**(a) **工序依赖图** G_T，**顶点是工序，走廊阻断找不到作为起点的工序**；(b) **预约依赖图** G_R，**顶点是走廊占用，从与阻断时段重叠的占用出发，沿四类依赖边（让行、同车、同工件、同机；定义 4）纳入后续占用。**

即车辆 k 在 $[t^s, t^e)$ 占用走廊 e，所属运输段为 u。同一走廊上任意两次不同占用必须满足 $[t^s, t^e) \cap [t^{s\prime}, t^{e\prime}) = \varnothing$。规划路径时，每条走廊只使用预约表 R 中尚未被占用的时段，因此新写入的占用不会与已有占用重叠，预约表始终满足上述排他约束，无需事后再做冲突检测。

时间窗路由是后文改路与各对照方法共用的基础模块：给定起点、终点、最早出发时刻与当前预约表 R，它返回一条在各走廊上的占用时段都不与已有占用重叠的路径，以及进入各走廊的时刻。实现上采用时间窗 Dijkstra [20]。本文不改动这一模块，只改变*哪些*运输段交给它重新规划。van Os 等对节点做预约 [23]，Lyu 等另加节点容量约束 [9]；本文只对走廊（路网中的边）做预约，节点不设预约，因此本文结果不能与二者报告的数值直接比较（第 7.1 节）。

3.1.3 **工序依赖图与预约依赖图.** 工序依赖图 $G_T = (T_T, E_T)$（文献中亦称任务图，下标 T 取自 task）以工序为顶点，有向边表示两道工序之间的先后约束，分两类：*工件先后*，即同一工件的工序 O_{ji} 指向它的下一道工序 $O_{j,i+1}$；*同机先后*，即同一台机器上的工序指向在该机器上紧随其后加工的工序。G_T 中没有走廊占用，因而也不含占用之间的让行关系。预约依赖图 $G_R = (R, E_R)$ 以预约表 R 中的走廊占用为顶点，有向边 $r \to r'$ 表示 r 失效可能迫使 r' 改路，分四类（定义 4）：同一走廊上前后衔接的两次占用之间的*让行*边，同一车辆相邻两次占用之间的*同车*边，以及把 G_T 中的工件先后与同机先后关系对应到运输占用上得到的*同工件*边与*同机*边。这里，一道工序的*运输占用*是指为该工序搬运工件的运输段所产生的走廊占用。两张图的顶点集不相交：一边是加工，一边是通行。因此存在一类只改变走廊可通行时段、不改变任何工序参数的扰动（如走廊阻断）：按 G_T 划界时找不到作为起点的工序；而在 G_R 上，只要原计划中有走廊占用与受扰时段重叠（这正是原排程因此不可行的情形），这些占用就构成非空的起点集合，划界可以由此开始。图 2 对照这两张图；图 1 给出走廊阻断下的后果；表 2 汇总本节及后文用到的记号。

3.2 执行期假设

执行期约定与物理设定如下。其中 A1–A4 由全部对照方法共用，以便把结果归因到重调度范围如何划定，而不是模型差异；A5 只约束本文方法，作为对照的全局重调度还可以改变机器指派与加工顺序（见 A5 末句）：

表 2. 本文常用记号

符号	含义
$G=(V,E),\tau_e$	走廊路网；走廊 e 的通行时间
v_0, $\mathrm{loc}(m)$	装卸站；机械臂 m 所在节点
x,π,y	机器指派、加工顺序、车辆指派
$R, r=(e,[t^s,t^e),k,u)$	预约表；一条半开占用（车辆 k，段标签 u）
G_T / G_R	工序依赖图 / 预约依赖图
$\mathcal{S}^0,\mathcal{S}'$	扰动前排程、恢复后排程
$\mathcal{D}$, t_{now}	扰动事件、发生（决策）时刻
T_{direct}, T_{impact}	工序依赖图上直接受扰的工序 / θ-步纳入范围
$\mathrm{Src}(\mathcal{D}),U$	作为起点的占用；允许改写的占用集合
$\mathrm{Cl}(\cdot)$	预约影响集（由 STRC 得到）
φ	受扰未来工序比例（规模扫描）

A1. 初始排程可行。 $\mathcal{S}^0$ 在扰动前通过无冲突校验；预约表与工序时间表一致。

A2. 已完成占用不回溯。只改写 $t^e > t_{\mathrm{now}}$ 的尚未结束占用与未完工工序；$t^e \le t_{\mathrm{now}}$ 的占用字段不可改写。

A3. 单次、完全观测。扰动在 t_{now} 被完整观测，且不与其它扰动叠加。本文不研究以下情形：多个扰动相继到达；扰动的信息（如阻断持续时间）在发生时只能估计且可能有误；随新信息到达而反复重调度。

A4. 走廊排他、缓冲充裕、车队同构单载。走廊仍为排他半开资源，阻断以附加占用的形式注入路由层，即在该走廊上写入一段不准通行的时段，下称阻断时段。有限缓冲、充电、加减速与装卸时长不建模；全部车辆在计划开始时空载停于 v_0。

A5. 允许的恢复动作。 B 类扰动保持原机器指派 x 与加工顺序 π，只改路；失败时扩大允许改写的占用集合。A 类故障额外允许：车辆故障把故障车从车队摘除并换车；机械臂故障把未完工工序改派到其它可行机。加工顺序 π 始终不变，不重开搜索。实验中的全局重调度对照可以改 x 与 π（第 5.3 节）。

3.3 扰动二分

定义 1 (A/B 类扰动). *若扰动直接使某些工序的指派或可执行性失效（机械臂故障、显著工时偏差、插单、车辆故障等），称其为* ***A 类****；若扰动只使某些走廊在某段时间内不可用、而不使任何工序不可执行（走廊阻断、走廊通行变慢等），称其为* ***B 类****。*

注 1 (车辆故障为何属于 A 类). *车辆故障不改动任何工序的机器指派，乍看应与走廊阻断同类。但定义 1 的判据是可执行性而非指派：车辆发生故障后，它承运的工序在原计划下无法开工。只要方法记录了每辆车承运哪些工序，这些工序就可以作为工序依赖图上的起点，按工序划界得到的范围不为空，因此车辆故障归 A 类。若方法只记录工序与机器（AOR 的常见实现即如此），车辆故障同样会得到空范围；第 6.8 节的对照方法记录了这一对应关系。*

定义 2 (工序依赖图上的直接集与影响域). *对扰动 $\mathcal{D}$，令 $T_{\mathrm{direct}}(\mathcal{D})$ 为因机械臂故障、车辆故障或工序失效而无法按原计划执行的工序集合。对 B 类走廊扰动约定 $T_{\mathrm{direct}}=\emptyset$。在 G_T 上从 T_{direct} 出发，沿工件后继边与同机后继边最多走 θ 步，所能到达的工序（含 T_{direct} 本身）记为 $T_{\mathrm{impact}}(\mathcal{D},\theta)$。*

定义 3 (作为起点的占用). *起点集 $\mathrm{Src}(\mathcal{D})\subseteq R$ 按扰动类型给出。各类型都只取 $r.t^e > t_{\mathrm{now}}$ 的占用，即在决策时刻尚未结束的占用；已经结束的占用不作为起点。*

(a) **走廊阻断/通行变慢** $\mathcal{D}=(e,[t_a,t_b))$：$\{r : r.e=e,\ [r.t^s,r.t^e)\cap[t_a,t_b)\neq\emptyset\}$，即走廊 e 上与受扰时段重叠的占用。有多处短暂阻断时，取各阻断时段各自起点占用的并集。通行变慢与阻断共用这条起点规则：原排程

按未变慢的通行时间写出，与受扰时段重叠的占用在两种扰动下都无法按原时刻完成，故二者给出同一个预约影响集。二者只在改路时不同：阻断时段内该走廊不可通行；通行变慢时走廊仍可通行，但落在受扰时段内的那部分通行时间按倍率加长。

(b) **车辆故障** $\mathcal{D} = (k)$：该车在 t_{now} 之后的全部占用 $\{r : r.k = k\}$。

(c) **机械臂故障** $\mathcal{D} = (m, F)$,F 为该机上尚未完工的工序：对每个工件取 F 中最小工序号 $i_{\min}(j)$，起点占用为各工件自该工序起（含）的全部尚未结束占用。之所以沿工件链向后取，而不只取该工序自身的占用，是因为该工序的进站运输可能已在 t_{now} 前结束：若只取自身，起点占用会变空或偏少，预约影响集覆盖的工序反而少于工序依赖图上的影响域，两种划界的比较就不公平——差别来自起点取法，而不是划界对象。

命题 1 (B 类上工序依赖图给出空范围). 设 $\mathcal{D}$ 为走廊阻断。若采用定义 2 对 B 类的约定，则 $T_{\text{direct}}(\mathcal{D}) = \emptyset$，从而对任意有限 $\theta \geq 0$ 有 $T_{\text{impact}}(\mathcal{D}, \theta) = \emptyset$。若此外 $\text{Src}(\mathcal{D}) \neq \emptyset$，则 $\mathcal{S}^0$ 在 $\mathcal{D}$ 下不可行。

证明. 走廊阻断只使走廊在某段时间内不可用而不使任何工序不可执行，故按定义 1 属 B 类。这一步用到两条前提：阻断时段 $[t_a, t_b)$ 有限，且由假设 A4 缓冲充裕，车辆可以在节点上等到阻断结束后再通行；因而即使阻断期内路网被割断，也没有工序变得不可执行，只是被推迟。若阻断无限期、或缓冲不足以支持等待，该扰动按定义 1 就该归 A 类，不在本命题范围内。既属 B 类，再由定义 2 对 B 类的约定，$T_{\text{direct}} = \emptyset$。$\theta$-步纳入的起点为空，故 $T_{\text{impact}} = \emptyset$。若存在 $r \in \text{Src}(\mathcal{D})$，则 r 的占用区间与阻断时段相交。将 $\mathcal{D}$ 解释为该走廊上的强制占用后，r 与阻断时段在同一排他资源上时间重叠，违背假设 A4 的走廊排他约束，故 $\mathcal{S}^0$ 不可行。 □

命题 1 是问题层面上的反例。前半句由定义 2 对 B 类的约定直接得到（工序依赖图按定义不含走廊占用之间的让行关系，因而无法刻画走廊阻断）；非平凡的是后半句——起点占用非空时原排程已经不可行，因而「工序依赖图上范围为空 ⇒ 无需重调度」在用时间窗预约走廊的设定下不成立。该反例不依赖任何修复算法；第 6.2 节在 50/50 对上逐一验证了这一情形。

3.4 恢复问题

问题 1 (按预约表划定范围的有界重调度). 给定可行排程 $\mathcal{S}^0$、扰动 $\mathcal{D}$ 与时刻 t_{now}，求 $\mathcal{S}'$，使得：*(i)* $\mathcal{S}'$ 在 $\mathcal{D}$ 下通过无冲突校验；*(ii)* 未纳入改写范围的占用，其走廊、时段与车辆尽可能与 $\mathcal{S}^0$ 一致；*(iii)* 算法耗时受预算约束。次级目标为完工时间 $C_{\max}(\mathcal{S}')$。

有界重调度拆为两步：先划定允许改写的占用集合 $U \subseteq R$；再只对 U 内运输改路，集外占用保持原计划；后一步称为有界修复（第 5 节）。本文用 Cl(Src) 给出 U；对照方法的 U 如何取，见第 4 节。

4 时空预约闭包

本节确定允许改写的占用集合 U：给定预约表 R 与起点占用集 $\text{Src}(\mathcal{D})$（定义 3），在占用之间建立依赖图，取从起点出发沿边可达的全部占用，即 $U = \text{Cl}(\text{Src})$。如何在 U 内改路见第 5 节。

4.1 预约依赖边

定义 4 (预约依赖图). 顶点为 R 中的走廊占用。有向边 $r \to r'$ 表示：r 一旦改写，r' 可能也必须改写。边分四类：

(1) **让行边**：同一走廊、不同车辆的两条占用，时间重叠或首尾相接（$r.t^e = r'.t^s$），由进入时刻 t^s 较早者指向较晚者；

(2) **同车后继**：同一车辆按时间先后相邻的两条占用，前一条指向后一条；

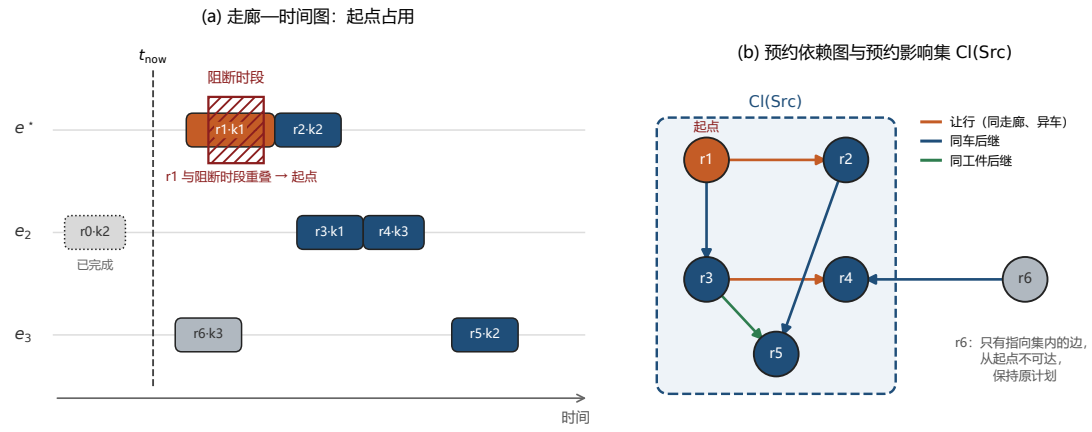


Fig. 3. 预约影响集的构造示意（同一组占用画在两幅中）。(a) 走廊—时间图：每个色块是一条走廊占用，标注为「占用·车辆」；只有 r_1 与走廊 e^* 的阻断时段重叠，成为起点；r_0 在 t_{now} 前已完成，不参与划界。(b) 这些占用之间的依赖边（颜色区分类型）：从 r_1 出发沿边可达 r_2–r_5，它们与 r_1 构成预约影响集 Cl(Src)（虚线框）。r_6 只有指向集内的边，从起点不可达，留在集外并保持原计划；集内没有指向集外的边。

(3) **同工件后继**：同一工件工序 i 的运输占用指向工序 $i+1$ 的运输占用；

(4) **同机后继**：同一机器上相邻加工的两道工序，前一道的运输占用指向后一道的运输占用。

由此得到 $G_R = (R, E_R)$。

四类边的示意见图 2(b)。首尾相接的两条占用在排他语义下并不冲突，但前一条一旦延后，后一条就必须让路，因此也连让行边。边集是一种建模选择：它可能多纳入不必改写的占用，也可能漏掉未建模的依赖（注 2）。

定义 5 (预约影响集与时空预约闭包). 给定起点占用集 $S \subseteq R$ 与时刻 t_{now}，记决策时刻尚未结束的占用为

$$R^\circ = \{ r \in R : r.t^e > t_{\text{now}} \},$$

令 G_R° 为只保留 R° 中的占用及它们之间的边所得的子图。预约影响集定义为

$$\text{Cl}(S) = \{ v \in R^\circ : \exists s \in S \cap R^\circ \text{ 使 } s \to^* v \text{ 于 } G_R^\circ \},$$

即从尚未结束的起点占用出发，沿边可达的全部尚未结束的占用（含起点自身）。计算 $\text{Cl}(S)$ 的过程称为时空预约闭包（*STRC*）。

用邻接表存储 G_R° 时，从起点做一次广度优先搜索即可得到 $\text{Cl}(S)$，耗时为 $O(|R^\circ| + |E_R^\circ|)$，其中 E_R° 为 G_R° 的边集。实验没有单独测量这一步的耗时，它包含在第 6.9 节报告的总耗时内。

图 3 用一个小例子说明这一构造。

按工序依赖图划界的对照方法取 U 为 T_{impact} 中各工序在 t_{now} 时尚未结束的运输占用；由命题 1，B 类扰动下这一集合为空。本文方法取 $U = \text{Cl}(\text{Src})$。实验中二者分别记为 R1 与 R2。

4.2 闭合性

命题 2 (结构闭合性). 对任意 $S \subseteq R$，在 G_R° 中不存在边 $u \to v$ 满足 $u \in \text{Cl}(S)$ 且 $v \in R^\circ \setminus \text{Cl}(S)$。因而，从 $\text{Cl}(S)$ 出发沿边无法到达影响集外、尚未结束的占用。

证明. 若存在这样的边 $u \to v$，则由 $u \in \mathrm{Cl}(S)$ 知存在尚未结束的起点占用 s 使 $s \to^* u$。拼接 $u \to v$ 得 $s \to^* v$，故按定义 5 应有 $v \in \mathrm{Cl}(S)$，矛盾。 □

命题 2 是定义的直接推论：边集一经固定，影响集对这些边就是封闭的。它是否覆盖了物理上真实存在的全部依赖，取决于边集的选取，见下面的注。

注 2 (预约影响集与最小允许改写集合). 闭合性只说明集内占用没有指向集外的依赖边；它既不保证影响集足以恢复可行，也不说明影响集已经足够小。下面分三点说明影响集与最小允许改写集合的关系。

其一，**最小允许改写集合依赖于所用的修复程序，而预约影响集不依赖**。说集合 U 最小，必须先指明「在何种修复能力下 U 足以恢复可行」：允许改派与改序时的最小 U，与只允许改路时的最小 U 不是同一个集合。预约影响集由 G_R 唯一确定，与修复程序无关，因此它不能、也不打算等于某一种修复程序下的最小集合。

其二，**四类边会多纳入不必改写的占用；已知的漏纳情形（首尾相接）已并入让行边**。让行边只要两条占用重叠就连边，并不判断后者是否真的只能等前者通过——若该处另有旁路，后者改道即可，本不必纳入改写；同车后继边也可能把延误传不到的链尾一并纳入。这两处使影响集偏大。为检验首尾相接这一处理是否到位，本文做差分试探：在与第 6.4 节相同的 50 对算例上，将影响集内的每条占用单独延后 1 个时间单位、其余不变，检查它是否与影响集之外同一走廊上的某条占用新产生时段重叠。每出现一次计为一次泄漏，即当前边集没有连出、但延后后会发生冲突的依赖。泄漏总数为 *0*，出现泄漏的对数为 *0/50*；其中 *44* 对的影响集外仍有占用，其余各对的影响集已包含全部尚未结束的占用。这一检验只针对首尾相接这一已知漏边，并不证明边集完备。

其三，**实测上影响集并不小**。走廊阻断下，预约影响集平均覆盖 t_{now} 之后尚未结束占用的约 *92%*（第 *6.8* 节）；与把尚未结束的占用全部纳入改写相比，少纳入的不足一成。有界修复耗时短，主要因为不重新启动搜索，而不是因为待改写的占用少得多；「局部」「有界」并不意味着改动范围很小。这不足一成的差别仍使改动比例平均降低 *0.064*，而算法耗时与完工时间几乎不变（第 *6.9* 节）。

因此，本文给出的是由结构唯一确定、可复现、对依赖边封闭的允许改写集合，且在本文全部实验中均足以恢复可行；逼近最小允许改写集合是另一个问题，留作后续工作。

4.3 影响集规模能否由路网结构预测

预约影响集的规模由 G_R 唯一确定：排程与路网给定后，没有可调参数。把算例从 $8 \times 4 \times 4$ 放大到 $20 \times 10 \times 10$ 时，影响集占尚未结束占用的比例在 0.78–0.91 之间，未随规模增大而上升（第 6.11 节）。因而「局部」的范围可以测量，但其规模并不小（注 2）。

影响集规模由排程与路网共同决定。实验考察了只由路网算出、不随排程变化的指标（装卸站最小割与漏斗占比），未发现它们能预测影响集规模：在外部路网上这些指标取值恒定，而同一张路网上换用不同机器配置时，影响集占比明显变化（第 6.6 节）。

5 预约影响集上的有界修复

图 4 总览本节流程：先按预约影响集划定允许改写的占用集合 U，再只为 U 内的占用重新规划路径与时段，集外占用保持原计划；仍不可行则扩大 U 再试。第 5.1 节给出改路程序及其保证，第 5.2 节给出范围扩展的规则，第 5.3 节说明对照方法。各方法共用同一套走廊路网、时间窗路由与无冲突校验。

5.1 保持原计划的单遍改路

给定允许改写的占用集合 U，单遍改路保持原车辆指派与机器指派，按原加工顺序逐个重新安排 U 内的运输。改路以单条占用为粒度：车辆在 t_{now} 前已通过的那部分行程属于已结束的占用，原样预提交，改路只从车辆当

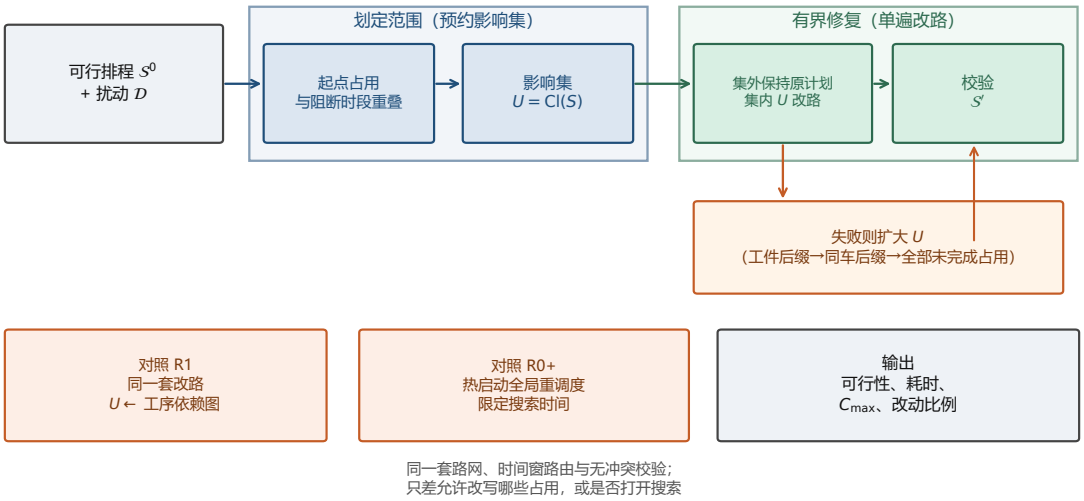


Fig. 4. 按预约影响集划界的有界修复。上排：由起点占用得到影响集并改路；中排：失败后扩大允许改写的范围；下排：按工序依赖图划界的对照与热启动全局重调度对照，以及输出指标。耗时同为毫秒量级的另外三种对照方法（RS、RD、RA）未画出，见第 6.9 节。

时所在的节点开始规划。算法 1 给出这一基本操作；算法 2（第 5.2 节）在其外层计算预约影响集，失败时扩大 U。

按复杂度，划界只需一次广度优先搜索（定义 5）；而一次 ReplayReroute 要为 U 内每个运输段各调用一次时间窗最短路，耗时主要来自后者。改路过程不启动任何种群搜索。第 6.4 节的对照固定算法 1，只替换 U 的来源（工序依赖图或预约影响集），这样两种划界方法可行率的差别只能来自划界对象。

命题 3 (集外字段不变的充分条件). 设允许改写的占用集合 $U = \mathrm{Cl}(S)$，按算法 1 执行单遍改路，且满足：*(i)* 阻断时段已写入路由层；*(ii)* 每条 $r \in R^{\circ} \setminus U$ 的原字段 $(e, [t^s, t^e), k)$ 都已预提交到新预约表，且预提交成功；*(iii)* 时间窗路由只为 U 内的占用重新规划；*(iv)* 最终排程通过无冲突校验。则在恢复后的预约表 R' 中，每条 $r \in R^{\circ} \setminus U$ 的走廊、时段与车辆都与 S^0 一致。本命题以单条占用为单位成立，而不是以整道运输为单位：一道运输可能一部分占用在 U 内、另一部分在 U 外，此时只有 U 外的那部分受本命题保护。

证明. 由 (ii)，每条集外占用在重放开始前已按原区间写入新表，且没有与阻断时段冲突（否则预提交失败）。由 (iii)，重放只为 U 内的占用提交新占用，不删除也不改写已提交的集外占用。因此每条集外占用在新表中的 $(e, [t^s, t^e), k)$ 与预提交值相同，即与 S^0 相同。(iv) 的校验只排除新占用与已有占用之间的冲突，不改变已提交的字段。 □

前提 (i)–(iv) 是改路程序自身的执行条件，可在实现中逐条检查；此外，命题还依赖假设 A1（初始排程可行，否则集外原字段之间就可能互相冲突、预提交必然失败）、A2（已结束占用不回溯）与 A4（走廊排他）。

注 3 (命题 3 的前提在何种情况下不成立). 有三种情况。第一，预提交失败：若某条集外占用与阻断时段重叠，前提 *(ii)* 不成立，说明起点集 S 没有覆盖全部与阻断时段重叠的占用，即起点集不完备。第二，范围扩展：若改路失败并触发第 5.2 节的范围扩展，允许改写的集合变为 $U' \supsetneq U$，命题只对最终仍在集外的占用成立。范围扩展意味着优先保证可行，而不是保证改写最少；发生范围扩展的情况在实验中单独报告。第三，实现粒度

算法 1 单遍改路 ReplayReroute

Require: 排程 $\mathcal{S}^0 = (x, \pi, y, R)$，扰动 $\mathcal{D}$，允许改写集合 U，时刻 t_{now}
Ensure: 排程 $\mathcal{S}'$ 或失败

将 $\mathcal{D}$ 写入路由层 ▷ 阻断：写入阻断时段；通行变慢：窗内通行时间按倍率加长
取空表 R'
for 每条不在 U 内的占用（其中包括全部已结束的占用） **do**
    将原字段 $(e, [t^s, t^e), k)$ 预提交到 R' ▷ 已结束的占用必须原样写入，遗漏即等于改写它们
    if 与阻断时段冲突 **then**
        return 失败 ▷ 起点占用未覆盖与阻断时段重叠的占用
    end if
end for
for 每条 $r \in U$，按原加工顺序 **do**
    从该车辆在 t_{now} 所在的节点开始规划 ▷ 已通过的行程已在上一步预提交，不重复提交
    在当前 R' 上调用时间窗路由，提交新占用
    按 π 更新相应机器的空闲时刻与工件的就绪时刻
end for
独立校验：无冲突、无占用落入阻断时段、通行变慢时窗内通行时间已按倍率加长、集外字段未改
if 校验通过 **then**
    return $\mathcal{S}' = (x, \pi, y, R')$
else
    return 失败
end if

过粗：前提 *(iii)* 要求时间窗路由只为 U 内的占用重新规划。若实现以比占用更粗的单位（例如整道运输、整个工序）决定是否改路，U 外的占用会被一并改写，命题的结论随之不成立。因此实现必须以单条占用为单位决定是否改路。粒度还须细到路段：对跨越 t_{now} 的运输段，已通过的路段原样保留，改路从车辆在 t_{now} 时所在的节点开始；否则会改写已结束的占用，违反假设 *A2*。由于这些路段在 U 内，集外字段的比对检查不出这种改写。

5.2 失败后扩大允许改写的范围

失败后逐步扩大范围的做法已有先例。Local Rescheduling [6] 按时间窗逐步放宽直至可行；AOR 则按工件后继与同机后继纳入后续工序（第 2.1 节）。本文沿预约依赖边扩大 U，而不是沿时间窗。这一步保证影响集过紧时仍能恢复可行，而不必立即转为全局搜索。E3 为比较两种划界的可行率，关闭了这一步（第 6.4 节）。

集外占用全部固定后，若 U 内的占用找不到可行的路径与时段（例如运输无法在工序要求的时刻送达），算法 1 返回失败。此时按下列顺序逐级扩大 U，并重试同一套改路程序，下称范围扩展：(i) $U \leftarrow U \cup \{$ 当前 U 内各工序的同工件后缀（该工件此后的全部运输占用）中尚未结束的占用 $\}$；(ii) $U \leftarrow U \cup \{$ 当前 U 内占用所属车辆的同车后缀（该车此后的全部占用）中尚未结束的占用 $\}$；(iii) $U \leftarrow \{ r \in R : r.t^e > t_{\text{now}} \}$，即把决策时刻尚未结束的占用全部纳入改写，这正是对照方法 RA 的允许改写集合（第 5.3 节）。范围扩展后仍是确定性的单遍改路。在启用范围扩展的实验中（扰动规模扫描与 E8），全部算例–随机种子对都恢复可行，算法耗时保持在毫秒至十余毫秒量级（第 6.10、6.11 节）。

算法 2 把第 5.1 节与本节合起来写出，与图 4 对应。

算法 2 基于预约影响集的有界修复（单遍改路 + 范围扩展）

Require: 排程 $\mathcal{S}^0$，扰动 $\mathcal{D}$，时刻 t_{now}
Ensure: 排程 $\mathcal{S}'$ 或失败
$S \leftarrow \text{Src}(\mathcal{D}); U \leftarrow \text{Cl}(S)$
for 轮次 $= 0, 1, 2, 3$ **do**
  $\mathcal{S}' \leftarrow \textsc{ReplayReroute}(\mathcal{S}^0, \mathcal{D}, U, t_{\text{now}})$
  if $\mathcal{S}'$ 可行 **then**
    return $\mathcal{S}'$
  end if
  按范围扩展的规则更新 U ▷ 工件后缀 / 同车后缀 / 全部尚未结束
end for
return 失败

5.3 对照方法

全局重调度对照（记 R0+）在相同禁行约束下、在限定时间内重新搜索。R0+ 用遗传算法搜索，把 (x, π) 编码为一条染色体；一次*解码*指：按该染色体的指派与加工顺序，在已写入阻断时段的预约表上从头为全部运输调用第 3.1.2 节的时间窗路由，得到新的 R 与 $C_{\max}$。R0+ 反复解码并允许改变染色体。它的解码不改写已经结束的占用：这些占用预提交，只从 t_{now} 起排定尚未开始的工序（第 6.7 节）。R0+ 采用热启动，即把原排程作为初始种群中的一个个体，以免丢掉原排程已有的指派与路径信息。R0+ 可改派与改序，因而更可能缩短 $C_{\max}$，但算法耗时由预算决定；两者在完工时间与算法耗时上的比较见第 6.7 节。

仅与 R0+ 对照还留下一个问题：在毫秒量级的耗时内，按影响集划界是否必要？为此本文实现三种不按影响集限制范围的做法：全局右移（RS）不判断哪些占用受影响，只把尚未结束的占用整体后推到阻断结束之后；原染色体重解码（RD）不改染色体，已结束的部分保持不变，其余按原染色体解码一次；**RA** 把 t_{now} 之后尚未结束的占用全部纳入改写，再执行与第 5.1 节 *完全相同*的改路程序，因而与本文方法只差「允许改写哪些占用」这一项，是唯一能单独检验预约影响集作用的对照。三者耗时均在毫秒量级，构成有界修复在*同一耗时量级内*的对照，定义与结果见第 6.9 节。

6 实验

6.1 实验设置

路网、路由与算例。下层路网、时间窗路由与独立校验器固定。扩样矩阵使用**五个算例**，规模记为「工件数 × 机器数 × AGV 数」：一个 $3 \times 3 \times 2$ 的小例，供逐步核对；一个 $8 \times 4 \times 4$ 的拥堵例；以及三个 $8 \times 4 \times 4$ 的脊式布局 funnel、high 与 mid。前两个为手工绘制；后三个由本文的算例生成器产生，生成参数包括拥堵档、异构度 h 与柔性度 f。脊式布局指一条主干走廊（脊）贯穿机器区，各机器经支线挂在脊上的两个枢纽处；装卸站位于脊的一端，经一条或两条并行出口接入 [19]。全部运输都要经过同一出口和同一段中段走廊，争用因此集中在这几条走廊上；拥堵例也是这种布局。三个脊式布局的规模、异构度与柔性度相同，唯一的差别是并行通道数：funnel 的装卸站出口与中段各一条，high 把出口增为两条，mid 再把中段增为两条。增删一条通道只改变可同时通过的车辆数，不改变路程，因此三者之间预约影响集规模的差异可归因于通道容量，而非问题尺寸。算例生成时的随机种子只决定算例本身，与下文决定扰动注入和搜索过程的运行随机种子相互独立。**运行随机种子取十个**，均在实验前一次性固定、未按结果挑选，依次为 42、7、2024、99、123、13、1、777、31415 与 8；E5 取其中前三个。五个算例与十个随机种子共组成 $5 \times 10 = 50$ 个算例–随机种子对，下文简称「对」。全文所有逐对比较、胜负计数与配对检验都以同一算例–随机种子对为单位：同一对上各方法读取同一份初始解、同

一段阻断与同一决策时刻。初始排程由第 3.1.2 节的时间窗路由按加工顺序重放得到，并通过独立无冲突校验；同一算例–随机种子对下各对照方法共享该初始解。对 $\mathcal{S}^0$ 只要求其在扰动前可行。

实现与运行环境。全部实验为纯 Python 标准库实现，不依赖第三方数值库；下层路网、时间窗路由与独立校验器复用与静态一体化工作同一份实现。单机单线程运行，不使用并行：Intel Xeon W-1270 （8 核 16 线程）、64 GB 内存、Windows 10，解释器为 CPython 3.7.6（64 位）。所有计时结果都在上述单机单线程条件下取得，更换硬件会使其整体平移，故本文的耗时结论均以比值或量级形式给出，不依赖绝对毫秒数。

第二批：采用已发表文献中的布局。上述五个算例的布局由本文构造，其中三个脊式布局的节点位置与路线长度相同，只是并行通道数不同、同时能通过的车辆数不同。为检验结论是否依赖这几张特定布局，另外运行一批 5 个算例：路网的网格尺寸、装卸站与各机器所在节点、网格中哪些相邻节点之间没有走廊，均逐项转录自 Lyu 等 [9] 附录 A 的布局图，转录结果再由脚本与 van Os 工具集所附的机读编码逐项核对。机器台数随布局而定，为 4 至 8 台。随机种子仍取同样十个，共 50 个算例–随机种子对。两批算例的工件数、AGV 数、每工件工序数、h、f 以及运输/加工时长比的标定目标一律相同，因此两批结果之差只能来自布局本身。

该附录按顺序给出六张布局（下称布局 1–6），依次含 3 至 8 台机器。本文对所有算例固定柔性度 $f = 0.6$；在只有 3 台机器时，每道工序平均只有 1.8 台候选机器，达不到「每道工序至少两台候选机器」的要求。若只为这一张布局放宽 f，又会多引入一个变量，所以未采用布局 1。

其余五张布局**只落在 2 张不同路网上**。布局 2、3（4、5 台机器）共用同一张 4×4 网格：16 个节点，相邻节点之间本应有 24 条走廊，布局图中有 1 处相邻节点之间没有走廊，实有 23 条。布局 4–6（6、7、8 台机器）共用同一张 5×5 网格：25 个节点，本应有 40 条走廊，有 2 处没有，实有 38 条。五张布局之间的差别在于机器台数与位置，而非路网形状。因此第 6.6 节分析这批数据时，**路网形状这一维的独立样本数是 2 而非 5**，机器配置（台数与位置）这一维则有 5 个；下文的论断均按此划分给出。

这批算例与 Lyu 等发布的数据有三处不同。**其一，逐段行驶时间为本文补齐。**Lyu 等只给出了单个示例算例的逐段时长（且各段不等），附录 A 的布局未给出逐段时长；这里按「所有边等权」补齐，这与 van Os 模型中「单步时长为常量」的假设一致，只是时间单位的标度不同。因此本批不与 Lyu 或 van Os 报告的数值作比较，只借用其路网。**其二，装货站与卸货站合并为一个装卸站。**原布局分设装货站与卸货站；本文沿用机器与 AGV 联合调度文献中常用的单装卸站设定 [2]（第 3 节），即工件从同一装卸站 v_0 出发、完工后返回该站。为此取车辆起始处的装货站作为 v_0，原卸货站作为普通网格节点保留。路网本身（网格、没有走廊的相邻节点对、各机器所在节点）不变，改变的只是每个工件首段与末段运输的端点。**其三，未采用其工件数据。**在已实测的公开算例族中，平均运输时长只占平均加工时长的百分之几，运输几乎不构成争用；若同时采用其工件数据，走廊争用以及以它为对象的全部现象都将无法观察。因此本批只采用其路网，工件数据仍按第一批的参数生成。

van Os 工具集中另有一组布局（标注为 Liu 2023），未纳入本批。这组布局允许车辆沿对角线行驶，而本文的路网只含上下左右相邻节点之间的走廊（第 3 节），两者的路网模型不同，故未纳入。

扰动。全部实验的决策时刻统一取 $t_{\text{now}} = 0.35\, C_{\max}(\mathcal{S}^0)$。该值在实验前固定：此时约四成占用已结束、六成尚未结束，既有须保持不变的已执行部分，也有足够的待修复部分；本文未对该取值做敏感性分析。各实验的扰动如下。E1–E3、E5 与 E7 阻断一条繁忙走廊，即在 t_{now} 时尚未结束的占用最多的走廊；阻断时段是一段连续的时间窗，从该走廊上这些占用中最早的开始时刻持续到最晚的结束时刻，因而覆盖它们全部。下文把这种扰动称为*繁忙走廊阻断*。E4 阻断装卸站最小割中的走廊（第 6.6 节）。E6 在同一决策时刻另加三类扰动（第 6.8 节）。扰动规模扫描先把在 t_{now} 时仍有尚未结束占用的工序，按各自最早一条这类占用的开始时刻排序，取前 φ 比例作为受扰工序；这些工序的每条尚未结束的占用各自生成一段阻断，阻断的走廊与时段就是该占用本身。各段阻断不合并成一段长时间窗，否则会同时盖住其间未被选中工序的占用。

对照方法。共六种方法，按与本文方法的差别由小到大列出：

- **R2（本文方法，STRC）**：按预约影响集划界，再做单遍改路。范围扩展只在扰动规模扫描与 E8 中开启，E1–E3、E5 与 E7 中关闭。
- **R1**：按工序依赖图上的影响域划界，再做与 R2 相同的单遍改路，全文取 $\theta = 2$。θ 是在工序依赖图上沿工件内工序先后关系向后扩张的最大跳数，它只影响 A 类扰动下 $|U_{R1}|$ 的大小；B 类扰动不直接作用于任何工序，起点工序集为空，无论扩张多少跳，结果都是空集（命题 1）。R1 与 R2 只差划界对象。
- **RA**：不按影响集限制范围，把 t_{now} 之后尚未结束的占用全部纳入改写，改路程序与 R2 完全相同，用于单独检验预约影响集的作用（第 6.9 节）。
- **RS**：全局右移。不划界也不改路，只把尚未结束的占用整体推到阻断结束之后（第 6.9 节）。
- **RD**：原染色体重解码。染色体不变，按固定前缀解码一次，是本实现中耗时最短的全局重算（第 6.9 节）。
- **R0+**：以原排程为起点、在限定时间内重新搜索的全局重调度。采用遗传算法，种群 40，原排程对应的染色体置入初始种群；解码采用与 RD 相同的固定前缀解码，t_{now} 之前的占用不可改写。不设代数上限与早停阈值，只按挂钟预算终止，预算取值见第 6.7 节。

五种对照与 R2 的差别逐级扩大：R1 只差划界对象，RA 只差是否划界，RS 既不划界也不改路，RD 与 R0+ 重算全局；RD 按原染色体解码一次，R0+ 在预算内反复搜索。

R1 的划界规则取自第 2.1 节所述的工序级局部重调度做法，由本文重新实现。逐条对应关系是：「受影响工序集取直接受扰工序，再沿工件内工序先后关系向后扩张」这一条划界规则对应 AOR [18]；而以跳数 θ 截断扩张、以及把工序集映射为须释放的走廊预约两项由本文写定——原文的设定里没有预约表，这一映射在原文中无对应物；改路与重放部件则与 R2 共用同一份实现，以保证两者只差划界对象。因此 R1 是将该划界规则移植到本文预约设定下的对照，其结果反映该规则在此设定下的表现，而非原方法在其自身设定下的表现。

预算口径。各方法不在同一预算档上比较：R1、R2、RS、RD、RA 均为一次确定性修复，运行结束即终止，耗时由算例规模与影响集大小决定而非由预算设定；只有 R0+ 按挂钟预算终止。因此涉及 R0+ 的结果都是在给定预算下得到的；其余五种方法都运行一次即终止，彼此可以直接比较。挂钟耗时按批次报告，不跨批次比较。

两批算例的分工。两批并非同一组证据的两次采样，它们回答的问题不同。主批次回答现象是否存在、机制能否被单独识别：B 类扰动下按工序依赖图划界得到空范围而原排程不可行（第 6.2 节）、预约影响集对该范围的包含与集外字段的不变性（第 6.3 节）、同一套改路程序下只替换划界对象的对照（第 6.4 节）。这三项都要求完全控制算例参数，尤其是决定走廊争用是否成立的运输/加工时长比，而公开算例族无法提供这种控制。第二批回答另一个问题：这些现象是否只在本文构造的布局上成立。它只用于检验主批次的结果在 Lyu 等的布局上能否复现（见第 6.12 节「外部布局批次的复现」），不产生新的性能对比结论。

两批合并分析的只有两处。其一是 E4：主批次的三张同规模布局同属一族、几何差异很小，「指标过粗」与「算例几何差异不足」两种解释无法区分；而外部布局若没有本文布局上逐参数同口径的对照，也无从判断其影响集占比的变动幅度是否足够大。第二批用于区分这两种解释。其二是 E5：两批的 E5 设置与参数逐项相同，合并只为增加样本量。其余各节的结论由主批次单独给出，第二批只增强其外部效度。

统计口径。全文有两处推断性检验，均在第 6.9 节：R2 与 RA 在改动比例上的差（主检验），以及 R2 与 RD 在完工时间上的差。二者均采用 Wilcoxon 符号秩检验，双侧，配对单位为算例–随机种子对，即取同一对上两种方法结果之差。按 Holm 法对这两次检验校正后，各自的显著性判断不变；其余各处的逐对胜负、均值与中位数均为描述性统计，不作显著性断言。主检验配套报告 Hodges–Lehmann 效应量估计及其 95% 自助置信区间（百分位法，10^4 次重采样，固定随机种子），离散度按四分位距报告。其余各批次的表格只给均值、中位与计数，不含区间。

表 3. E1 走廊阻断下两种划界给出的范围（每算例 10 个随机种子，共 50 对）。「三条件通过」计命题 1 的三个条件同时成立的算例–随机种子对（$T_{\text{impact}} = 0$、起点占用非空、阻断后原排程不可行）。$|R|$ 为全表占用数，|Src| 为起点占用数，|Cl| 为预约影响集的占用数，Cl/$|R|$ 为预约影响集占全表的比例；均值取 10 个随机种子的平均。funnel、high、mid 为三个脊式布局：funnel 的装卸站出口与中段各一条通道，high 的出口为两条，mid 的出口与中段均为两条。

算例	随机种子	三条件通过	均 $\lvert R\rvert$	均 |Src|	均 |Cl|	Cl/$\lvert R\rvert$
小例	10	10/10	28.6	5.6	16.9	0.59
拥堵例	10	10/10	85.0	15.0	48.4	0.57
funnel	10	10/10	149.1	15.5	85.4	0.57
high	10	10/10	150.5	15.5	88.3	0.59
mid	10	10/10	147.6	15.0	82.8	0.56
合计	50	50/50	—	—	—	—

6.2 E1：工序依赖图给出空范围

本节检验命题 1 在扩样算例上是否成立。按第 6.1 节的设置，在 50 对上各阻断一条繁忙走廊，分别按工序依赖图与按预约影响集划界，并检查阻断后原排程是否仍可行。结果如表 3 与图 5（左）所示。

从表 3 可以看到，命题的三个条件在全部 50/50 对上同时成立：按工序依赖图得到的受影响工序集为空（$T_{\text{impact}} = 0$），起点占用与预约影响集非空，原排程在阻断后不可行。其中第一条由定义 2 对 B 类的约定直接得到；有实质内容的是第三条，即原排程确实已经不可行。因此在这些对上，「工序依赖图给出空范围，故无需重调度」这一推断都不成立。

表中还可以看到，|Cl| 随算例变大而增大（16.9、48.4、82.8–88.3），这是占用总数变多的直接结果。Cl/$|R|$ 在五个算例上都落在 0.56–0.59 之间，算例之间的差别与同一算例内不同随机种子之间的差别同量级：逐算例的四分位距为 0.024–0.049，50 对合在一起的中位数为 0.571、极差为 0.500–0.655。

本节结果支撑贡献 C1：B 类扰动下，按工序依赖图划界得到空范围，而原排程已不可行。

6.3 E2：包含性

本节检验预约影响集的两条性质在实验中是否成立。**E2a，结构闭合**（命题 2）：没有依赖边从预约影响集内指向集外尚未结束的占用。这是图上的性质，由定义直接保证，检验的是实现与定义是否一致。**E2b，集外不变**（命题 3）：只在影响集内单遍改路后，集外每条占用的走廊、时段与车辆都保持原值。这是修复之后的性质，检验的是该命题的前提在实验中是否满足。该命题有一条前提要求：修复开始前，每条集外占用都能按原字段写回新的预约表（下称预提交）。若某条集外占用与阻断时段重叠，预提交就会失败，说明起点占用集漏掉了与阻断重叠的占用。

结果如表 4 所示。在全部 50 对上，影响集都是封闭的；只在影响集内单遍改路（不做范围扩展）都恢复了可行；集外占用逐条保持原值，即上述预提交失败的情况没有出现。由于跨越 t_{now} 的运输段在影响集内，E2b 不比对其已通过的路段；为此另行逐条核对 $t^e \le t_{\text{now}}$ 的占用，全部 50 对上均未被改写。

本节结果与 E1 一起支撑贡献 C1 与 C2 中关于划界对象的部分：预约影响集由依赖边集唯一确定、对集外封闭，只在其内改路即可恢复可行，且集外占用保持原值。

6.4 E3：边界消融

本节检验：在同一套改路程序下只更换划界对象，恢复可行率是否随之改变。R1 与 R2 使用同一份初始解、同一个决策时刻、同一批随机种子与同一段阻断时段，都只做单遍改路，**唯一的差别是允许改写哪些占用**。本节

表 4. E2 包含性（每算例 10 个随机种子）。E2a：没有依赖边从预约影响集内指向集外尚未结束的占用；「单遍改路可行」：只在影响集内改路、不做范围扩展时恢复可行；E2b：集外占用的走廊、时段与车辆逐条不变。funnel、high、mid 的含义见表 3。

算例	E2a 闭合	单遍改路可行	E2b 集外逐条不变
小例	10/10	10/10	10/10
拥堵例	10/10	10/10	10/10
funnel	10/10	10/10	10/10
high	10/10	10/10	10/10
mid	10/10	10/10	10/10
合计	50/50	50/50	50/50

表 5. E3 边界消融（不做范围扩展；每算例 10 个随机种子）。「R1 空、R2 非空」计 R1 的允许改写集合为空而 R2 非空的算例–随机种子对；末两列为 10 个随机种子的均值。funnel、high、mid 的含义见表 3。

算例	R1 空、R2 非空	R1 可行	R2 可行	均 $\lvert U_{R2}\rvert$	R2 耗时/ms
小例	10/10	0/10	10/10	16.9	0.66
拥堵例	10/10	0/10	10/10	48.4	2.84
funnel	10/10	0/10	10/10	85.4	5.09
high	10/10	0/10	10/10	88.3	3.40
mid	10/10	0/10	10/10	82.8	3.96
合计	50/50	0/50	50/50	—	—

关闭范围扩展：若开启，R1 可在扩大到全部尚未结束的占用后恢复可行，测得的就不再是划界对象的差别。结果如表 5 与图 5（右）所示。

从表 5 可以看到，在全部 50 对上，R1 的允许改写集合都为空而 R2 非空；R1 全部不可行，R2 全部恢复可行。R1 的允许改写集合为空时，单遍改路不改动任何占用，原排程原样保留，仍占用被阻断的走廊。因此，R1 不可行是因为它判定无需重调度，而不是修复做得不好。R2 单遍改路耗时的中位数为 3.2 ms，最大为 12.8 ms。

本文的走廊边权取等权，即每条走廊的通行时间相同。为检验上述结果是否依赖这一设定，另按三种非均匀边权在同一批 50 对上重新运行 E1 与 E3：其一，各走廊通行时间独立乘以对数正态因子（几何均值为 1）；其二，离装卸站越远的走廊越慢，最远处为原来的 1.8 倍；其三，各走廊随机取原通行时间的一半或两倍。三种边权下，E1 的三个条件仍在全部对上成立，R1 仍全部不可行；R2 在前两种边权下全部恢复可行，在第三种下有 1 对未能恢复（49/50）。可见，按工序依赖图划界得到空范围、R1 无法恢复可行这两点，都不随边权改变。

本节结果支撑贡献 C2 与 C3：只替换划界对象，恢复可行率就从 0/50 变为 50/50。

6.5 案例甘特图

E1–E3 给出的是 50 对上的计数与可行率。为在工序时间轴上直观展示「工序依赖图给出空范围、按预约影响集可以修复」的情形，图 6 给出一个具体例子（选小例是因为其甘特图能逐道工序看清）：算例为 $3 \times 3 \times 2$ 的小例（3 个工件、3 台机器、2 辆 AGV），运行随机种子为 42，扰动为繁忙走廊阻断，其余设置与 E3 相同。

该例上按工序依赖图得到的受影响工序为空（$T_{\text{impact}} = \varnothing$），而起点占用有 6 条、预约影响集有 18 条。图 6(a) 为扰动前原排程的机器甘特图：蓝色工序的运输占用落入影响集（将被改路），灰色工序在影响集外（预提交、保持原计划）；红色虚线为 t_{now}，浅红带为阻断时段。图 6(b) 为修复后：影响集内工序时间轴重排，排程在阻断下恢复可行，$C_{\max}$ ：57 → 86，耗时亚毫秒级。本图只说明局部改动的形态，不用于比较完工时间。

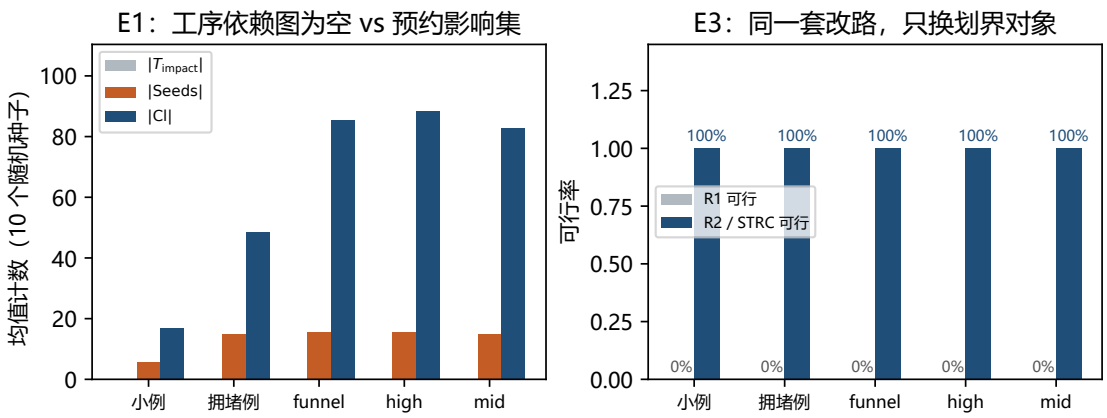


Fig. 5. E1 与 E3 的结果（每算例 10 个随机种子）。左：每算例按工序依赖图得到的受影响工序数、起点占用数与预约影响集的占用数，柱高为 10 个随机种子的均值；|Cl| 的逐算例四分位距为 2–8。右：单遍改路、不做范围扩展时 R1 与 R2 的恢复可行率，按算例–随机种子对计数。

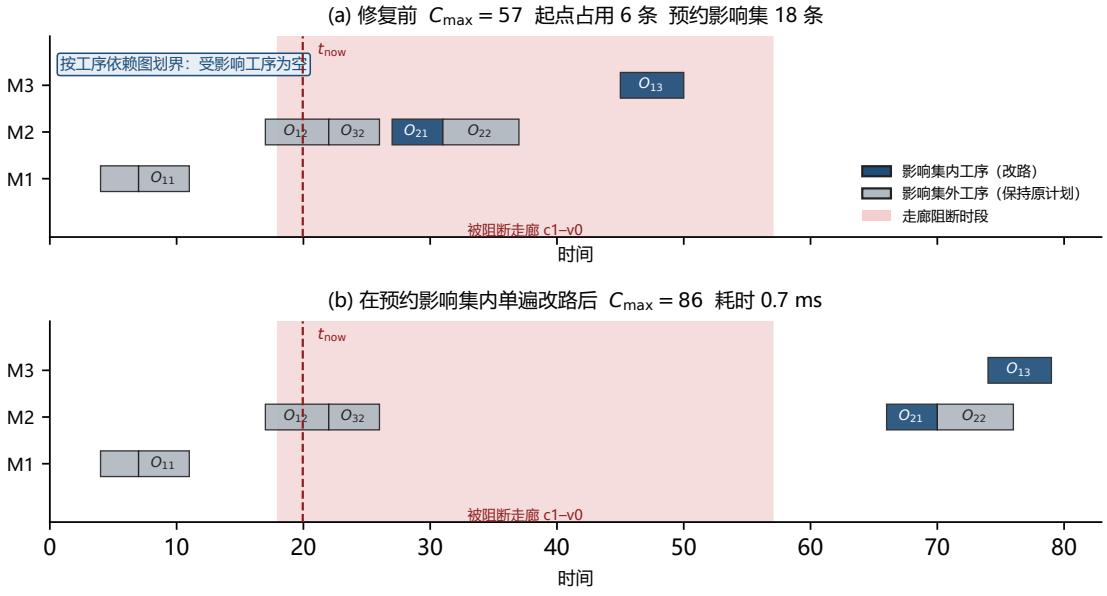


Fig. 6. 案例甘特图（3 × 3 × 2 小例，运行随机种子 42，繁忙走廊阻断）。(a) 阻断前原排程，按是否属于 Cl 着色；(b) 单遍改路后。工序依赖图上的影响域为空；蓝色工序对应影响集内运输被改路。本图仅示意局部改动的形态。

表 6. E4 静态路网指标与预约影响集占比（阻断 LU 割中的一条走廊；每张布局 10 个随机种子，Cl/|R| 取中位数）。全部算例的工件数、AGV 数与其余生成参数相同，只差路网与机器配置。「节点/走廊」为节点数与走廊数。「极差」为同一组内各布局影响集占比的最大值减最小值；4 × 4 网格一组只有 2 张布局、本文构造的一组只有 3 张，故极差只作粗略比较。

来源与网格	布局	机器	节点/走廊	LU 割	漏斗占比	Cl/|R|	极差
Lyu 等，4 × 4 网格	布局 2	4	16/23	2	0	0.512	
Lyu 等，4 × 4 网格	布局 3	5	16/23	2	0	0.305	0.207
Lyu 等，5 × 5 网格	布局 4	6	25/38	2	0	0.385	
Lyu 等，5 × 5 网格	布局 5	7	25/38	2	0	0.422	
Lyu 等，5 × 5 网格	布局 6	8	25/38	2	0	0.460	0.075
本文构造	funnel	4	9/8	1	0.286	0.491	
本文构造	high	4	10/10	2	0.286	0.497	
本文构造	mid	4	11/12	2	0.286	0.446	0.051

6.6 E4：路网结构指标能否预测影响集规模（探索性）

本节检验：只由路网算出、不随当前排程变化的指标，能否预测预约影响集的规模。考察三个指标：其一，*LU 割*，即使装卸站与全部机器断开所需切断的最少走廊数（装卸站最小割）；其二，*漏斗占比*，即从装卸站到各台机器的最短路中，所有路径都必须经过的走廊，其通行时间占平均路程时间的比例；其三，每节点走廊数，即走廊数与节点数之比。下文把这三个指标统称为*静态路网指标*。本节的扰动是阻断 LU 割中的一条走廊，其余设置同第 6.1 节；两批算例共 8 张布局，每张布局 10 个随机种子。结果如表 6 所示。

从表 6 最后三行可以看到，本文构造的三张脊式布局的影响集占比在 0.446–0.497 之间，极差只有 0.051；而它们的几何差异也很小（节点 9–11 个、走廊 8–12 条，LU 割只取 1 或 2）。「指标太粗、反映不出布局差异」与「这几张布局本身差别太小」两种解释都与此相符，**因此仅凭本文构造的三张布局无法判断静态路网指标是否有效**，需要借助第二批算例。

第二批的五张布局只落在两张网格上（第 6.1 节），所以只能用于考察机器配置这一维。在同一张 4 × 4 网格上，只把机器从 4 台换成 5 台（布局 2 换成布局 3），影响集占比的中位数就由 0.512 降到 0.305，相差 0.207；在同一张 5 × 5 网格上换三种机器配置（布局 4–6），相差 0.075。而本文构造的三张布局路网各不相同，彼此只相差 0.051；前两者约为它的四倍与一点五倍。这一比较并不对称：Lyu 等的布局同时改变了机器台数与位置，本文的三张布局只改变路网，且彼此几何差异很小，0.051 只是更换路网所引起变化的下界，故不能据此说机器配置的影响大于路网几何。**因此，路网不变、只改机器配置，影响集占比就会发生明显变化，机器配置这一维不可忽略。**

表中五个外部算例的 LU 割都是 2、漏斗占比都是 0。原因是两张网格都把装卸站放在网格角点：在只允许上下左右移动的网格上，角点只有两条走廊，故 LU 割固定为 2；装卸站到各台机器的最短路也不共用任何一条走廊，故漏斗占比为 0。van Os 工具集中的 Liu 2023 布局（因允许对角移动未纳入本批）把装卸站放在网格边的中点，出边数为 3。可见 LU 割在实际布局上只取少数几个由装卸站位置决定的值，不随路网几何连续变化。影响集占比在这些算例上变化了 0.207，而这两个指标取值不变，故无法解释这一变化。第三个指标（每节点走廊数）在整批上只取两个值（23/16 与 38/25），同样无法解释同一张网格内的变化。这些指标原本针对脊式布局中可调的装卸站出口瓶颈而设计，在实际布局上退化为常量。**因此，静态路网指标在 Lyu 等的布局上取值恒定，无法解释影响集规模的变化。**

结论：在第二批的 5 个算例与本文构造的三张布局上，静态路网指标都不能解释影响集规模；影响集规模更可能取决于运输需求在路网上的空间分布，而不是路网能提供多少条通路，具体的候选量见第 7.2 节「后续工作」。

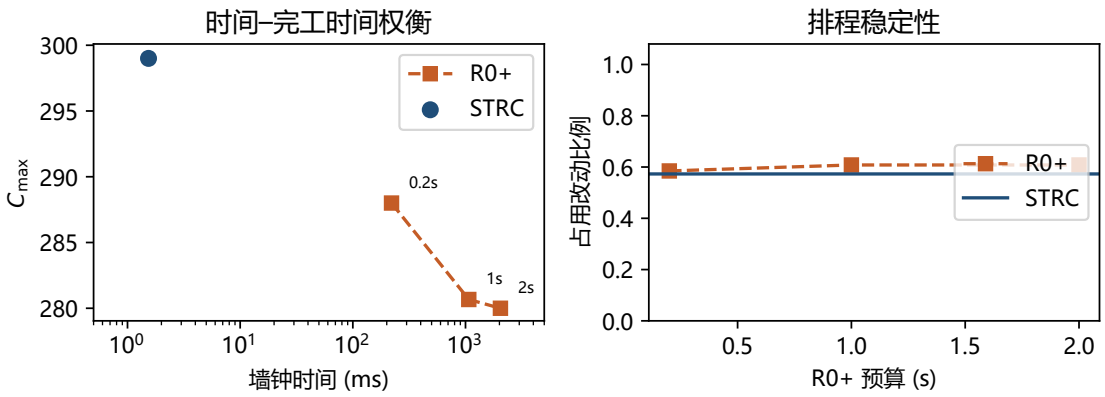


Fig. 7. E5：不改写已结束占用的 R0+ 预算扫描与有界修复结果（拥堵例，3 个随机种子均值）。左：算法耗时与 C_{max}；右：占用改动比例。

表 7. E5 与不改写已结束占用的热启动全局重调度（3 个随机种子均值）。本表只含本文构造的两个算例：2 算例 × 3 随机种子 × 3 预算点，独立的算例–随机种子对共 6 个；外部布局批次见第 6.12 节。「改动比例」为被改写的占用占全表之比。R2 在每对上只运行一次、不受预算影响，故同一算例三个预算行中的 R2 两列数值相同，均为同一次运行的结果。

算例	预算/s	R0+ C_{max}	R2 C_{max}	改动比例 R0+ / R2
拥堵例	0.2	288.0	299.0	0.58 / 0.57
	1.0	280.7	299.0	0.61 / 0.57
	2.0	280.0	299.0	0.61 / 0.57
小例	0.2	78.3	88.7	0.60 / 0.60
	1.0	78.3	88.7	0.60 / 0.60
	2.0	78.3	88.7	0.60 / 0.60

6.7 E5：与全局重调度的权衡

本小节检验是否存在交叉点，即预算紧时有界修复占优、预算放宽后热启动全局重调度在 C_{max} 上反超。对照方法为不改写已结束占用的热启动全局重调度 R0+，预算取 $\{0.2, 1, 2\}$ s；R2 在每对上只运行一次，结果与预算无关。算例分两批：本节给出本文构造的小例与拥堵例，随机种子 $\{42, 7, 2024\}$；第 6.12 节在两个外部布局上以同一组参数重复本节实验。两批合计 4 个算例、12 个算例–随机种子对、36 个预算点。

本文构造的两个算例的结果如表 7 与图 7 所示，从中可以看到四点。

第一，在每个预算点上 R0+ 的 C_{max} 都低于 R2，图 7 左的两条曲线不相交；连同外部布局批次，36 个预算点上 R0+ 胜/持平/负为 35/1/0。

第二，所测区间内不存在交叉点，且这一点可以推广到更大的预算。R2 的 C_{max} 不随预算变化；R0+ 报告的是逐代最优，随机种子固定后增加预算只是让同一条搜索轨迹多运行若干代，C_{max} 单调不增。因此两者的次序一旦在最小预算档上确定，增加预算便不会使其反转。在最小档上，12 对中 R0+ 有 11 对严格更优、1 对持平，没有一对更差；单调性与 R2 不随预算变化两点已在两批数据上逐对核对。需要区分的是：「增加预算不会使次序反转」由单调性保证，对任何算例都成立；「在最小预算上 R0+ 不差于 R2」只是在这 12 对上观测到的，换用其他算例未必成立。36 个预算点来自 4 个算例、12 对，同一对的三个预算点属于同一条搜索轨迹，彼此不独立，故本节不作显著性检验。

表 8. E6 扰动类型 × 划界对象（50 对/类，共 200 次测量）。通行变慢与阻断的起点占用相同，故两者给出同一个预约影响集，独立的划界测量为 150 次。$|R^{\circ}|$ 为 t_{now} 之后尚未结束的占用数。「R1 为空」计按工序依赖图划界判该扰动无需重调度的算例–随机种子对。末列不做范围扩展：B 类为单遍改路，A 类为换车或改派。

扰动	类	R1 为空	均 $\lvert U_{R1}\rvert$	均 $\lvert \mathrm{Cl}\rvert$	$\mathrm{Cl}/\lvert R^{\circ}\rvert$	R2 可行
走廊阻断	B	50/50	0.0	64.4	0.92	50/50
走廊通行变慢	B	50/50	0.0	64.4	0.92	50/50
车辆故障	A	0/50	31.1	60.6	0.85	50/50
机械臂故障	A	0/50	42.8	65.9	0.94	50/50

第三，上述推广不适用于更低的预算。R0+ 每评估一个候选至少要做一次解码，一次解码约需 4.16 ms（中位数）；种群为 40 个个体，仅评估初始种群就需约 0.17 s，已接近所测最小预算 0.2 s。因此预算低于 0.2 s 时，R0+ 可能连一代都评估不完，次序可能反转，而 R2 只需毫秒量级且在全部对上可行；本文未测试这一范围。

第四，有界修复的完工时间在全部预算点上均较差，但算法耗时低两至三个数量级（单遍改路 3.2 ms，对比 0.2–2 s 预算，见图 7 左）。这一耗时差来自单遍改路、不重新启动搜索，并非来自影响集划界；第 6.9 节（毫秒量级内的对照）以耗时相同的 RA 对照证实了这一点。两法的占用改动比例都约为 0.6（表 7 末列与图 7 右）。

上述完工时间比较中，两法都不改写已经结束的占用。R0+ 采用第 5.3 节的固定前缀解码；若在 t_{now} 时某辆车已空驶出发去取货，而该工件的前一道工序尚未完工、取货时刻须重新确定，R0+ 仍用原车辆，已驶过的路段原样保留，只重新规划 t_{now} 之后的部分。逐点检查表明，两个算例各 9 个预算点上 R0+ 改写已结束占用的次数均为 0，有界修复在相同的对上同样为零。因此两法的完工时间差距不包含「改写已结束的占用」带来的部分，而来自全局改派与改序相对单遍改路的优势。

结论：要求毫秒量级响应时，有界修复能够恢复可行；允许秒级停机时，不改写已结束占用的热启动全局重调度在完工时间上占优。预约影响集的作用不体现在完工时间上，其适用场景是要求毫秒量级响应的恢复。

6.8 E6：扰动类型与划界对象的交叉

E1–E3 只考察了走廊阻断一种扰动。本小节检验 E1–E3 的划界与修复结论在其他扰动类型下是否仍然成立：在同一 50 对上另加走廊通行变慢、车辆故障与机械臂故障三类。

测量内容。划界方面，四类扰动均报告工序依赖图上影响域的规模、预约影响集的规模，以及后者是否包含前者。修复方面，四类扰动均执行修复。B 类仍只改路：阻断时段写入预约表；通行变慢在变慢时段内使该走廊的通行时间加倍，时段外照常，走廊始终可通行。A 类额外允许换车或改派，不改变加工顺序；是否扩大允许改写的范围不影响其动作集合。

结果如表 8 所示，从中可以看到四点。

第一，两类 B 扰动按工序依赖图划界全部为空，两类 A 扰动全部非空，即 **A/B 二分在 150 次独立划界测量上无一例外**。

第二，在全部 100 次 A 类测量上 $U_{R1} \subseteq \mathrm{Cl}$，包含在 87/100 对上是严格的，其余 13 对两者相等。其中机械臂故障一行的包含按构造成立：定义 3(c) 的起点规则沿工件链取整条后缀，而 U_{R1} 以 $\theta = 2$ 截断（第 6.1 节），故该行只验证实现与定义一致；车辆故障一行的包含则是实测结果。允许改写的占用平均变多：车辆故障下，由按工序依赖图划界的 31.1 条增至按预约影响集划界的 60.6 条；机械臂故障下，由 42.8 条增至 65.9 条。**因此在 A 类扰动上，改用预约影响集划界不会遗漏工序依赖图已纳入的占用，代价是允许改写的范围更大。**

第三，通行变慢与阻断在全部 50 对上的影响集规模逐对相同。这是因为二者的起点规则相同（与受扰时段重叠、尚未结束的占用），按构造给出同一个预约影响集；按工序依赖图划界对通行变慢同样给出空范围。**因此通行变慢与阻断在划界上表现相同。**

表 9. E7 毫秒量级内对照（50 对，设置与 E3 相同：繁忙走廊阻断，R2 不做范围扩展）。四法均 50/50 可行。「已结束占用越界」计改写了已结束占用的算例–随机种子对；「允许改写」为该方法纳入改写的占用占 t_{now} 之后尚未结束占用的比例；末列为该方法对 R2 的 $C_{\max}$ 胜负。「平均退化」为 $(C_{\max}(\mathcal{S}') - C_{\max}(\mathcal{S}^0))/C_{\max}(\mathcal{S}^0)$ 在 50 对上的均值，其中 $\mathcal{S}^0$ 为扰动前的排程，$\mathcal{S}'$ 为修复后的排程。RA 与 R2 只差允许改写哪些占用，故这两行之差可全部归因于预约影响集。表 7 只含小例与拥堵例两个算例，与本表的 50 对不是同一组算例，故两表数值不宜直接比较。

方法	已结束占用越界	ms（中位）	平均退化	改动比例	允许改写	对 R2 赢/平/输
RS 全局右移	0/50	1.42	58.8%	0.627	—	7/16/27
RD 重解码	0/50	4.16	49.1%	0.618	—	22/14/14
RA 不按影响集划界	0/50	2.76	49.9%	0.604	1.00	0/46/4
R2 有界修复	0/50	2.74	49.8%	0.540	0.92	—

第四，四类扰动的修复全部可行（表 8 末列）：车辆故障采用换车，机械臂故障采用改派，两类 B 扰动单遍改路即全部可行，未触发范围扩展。通行变慢只使变慢时段内的通行时间加倍，走廊保持可通行，因此它与「整段禁行后绕路」在物理上仍有区别。**因此四类扰动下有界修复都能恢复可行。**

影响集规模。走廊阻断下，预约影响集包含了 t_{now} 时尚未结束占用中的约 92%；也就是说，与「把尚未结束的占用全部纳入改写」相比，它只少纳入了不到 10% 的占用。这一比例与 E1 中的 Cl/$|R|$（0.56–0.59）分母不同：后者以全表占用为分母，其中约四成在 t_{now} 前已结束。

结论：本节结果把 E1–E3 关于 C1、C2 的结论从走廊阻断推广到四类扰动。

6.9 E7：毫秒量级内的对照

E5 的对照是热启动种群搜索，预算 0.2–2 s。仅以它为对照会留下一个问题：「低两至三个数量级」这一结果，有多少来自预约影响集，又有多少只是因为对照方法耗时更长？本节补充耗时介于有界修复与热启动种群搜索之间的对照，并加入一个仅改变允许改写范围的对照。

- **RS（全局右移）。**不改路径、不改指派、不改加工顺序，把未完成占用整体后推一个延迟 Δ，Δ 取到刚好让受阻走廊上所有可动的占用落到阻断时段之后。不改写已经结束的占用（$t^e \le t_{\text{now}}$ 不可动），与 R2 相同；由于是统一平移，所有时间间隔保持不变或变大，而全部约束都是「≥」型，无需重新路由即得可行解。它代表「不按影响集划界」的一端，且完全不改路（第 2.1 节）。
- **RD（原染色体重解码）。**保持机器指派与加工顺序，在已写入阻断的路由层上执行与 R0+ 相同的固定前缀解码。它是本文实现中耗时最短的全局重算；由于不改变染色体，其耗时比 R0+ 短，改派与改序的自由度也更小。
- **RA（不按影响集限制范围，但保留改路）。**把 t_{now} 之后尚未结束的占用全部纳入改写，再执行与 R2 完全相同的改路重放。它代表「不按影响集划界」的另一端：与 RS 不同，它保留了改路，因而与 R2 的唯一区别是允许改写的占用集合。两者使用同一套改路程序，都不改写已经结束的占用，且面对同一段阻断。因此它是本文唯一能单独隔离预约影响集作用的对照，两法之差不含改路程序或实现上的差别。RA 的允许改写集合已是最大，无需范围扩展。

E3 比较的是两种不同的划界对象（工序依赖图与预约影响集），而 RA 回答的问题是：是否需要按影响集限制改写范围。

结果如表 9 所示，从中可以看到四点。

其一，RS 只需 1.42 ms，约为 R2 的一半，是给出可行方案的各法中唯一比有界修复更快的方法（按工序依赖图划界的 R1 更快，但其可行率仅为 0/50，不构成候选方法）。代价是完工时间平均退化 58.8%（R2 为 49.8%），逐对比较为 27/16/7（R2 赢/平/输），且改动的占用更多（0.627 对 0.540），因为统一平移改变了每一条尚未结束

占用的时刻。若要使右移有选择，即只推迟真正受影响的车辆而其余保持不变，就必须先确定哪些占用受影响，而这正是本文划定的范围。**因此，右移的速度来自放弃选择，代价是完工时间与改写量都更差。**

其二，RD 的耗时中位数为 4.16 ms，是 R2 的 1.5 倍。这一结果与搜索配置无关：任何基于解码的搜索，每评估一个候选至少需要一次解码。**因此，即使换成同一实现中耗时最短的全局重算，有界修复仍然更快**；E5 中「低两至三个数量级」并非由于对照方法耗时过长。

其三，全表中 R2 对 RD 为 14/14/22（赢/平/输），平均退化 49.8% 对 49.1%；配对差均值 +0.006，自助 95% 区间 $[-0.007, 0.019]$ 包含 0，Wilcoxon 符号秩双侧 $p = 0.12$，差别不显著。RD 在 0/50 对上改写了已结束的占用：若在 t_{now} 时某辆车已空驶出发去取货，而该工件的前一道工序尚未完工，RD 仍用原车辆，已驶过的路段原样保留，只重新规划之后的部分。因此 RD 在逐对胜负计数上的微弱优势来自同一染色体下全局改路相对单遍改路的优势，而非改写已结束的占用。RD 的改动比例为 0.618，高于 R2 的 0.540。**因此，在不改写已结束占用的前提下，RD 与 R2 的完工时间无法区分，但 RD 改写更多。**

其四，RA 与 R2 的唯一区别是允许改写的占用集合，因此这一对结果直接给出按影响集划界的收益。预约影响集平均只比全部尚未结束的占用少纳入 1–0.92 的尚未结束占用（各对取值为 0.80–1.00；在取值为 1.00 的 6 对上，影响集等于全部尚未结束占用，两法结果相同）。这不足一成的差别使改动比例从 0.604 降至 0.540：平均降低 0.064，中位数降低 0.029，逐对为 31/18/1（R2 更稳/持平/更差），Wilcoxon 符号秩双侧 $p = 8.8\times10^{-7}$。效应量的 Hodges–Lehmann 估计为 0.037，95% 自助置信区间为 $[0.018, 0.074]$；配对差均值 0.064 的区间为 $[0.039, 0.093]$。改动比例的四分位距在 R2 上为 0.073、在 RA 上为 0.062。由于 50 对中有 18 对两法恰好持平，配对差中位数的置信区间下端为 0。

剔除上述结果相同的 6 对后余 44 对，逐对为 31/12/1，平均降低 0.073（两种口径下 Wilcoxon 检验中差值非零的均为 32 对），即按全部 50 对报告的降幅偏保守。降低的方向高度一致，但幅度不大；在 R2 更差的唯一一对上，R2 多改写了 0.007。与此同时，两法的算法耗时几乎相同（2.74 对 2.76 ms），完工时间也几乎相同（比值均值 0.999，逐对 R2 赢/平/输为 4/46/0，从未更差，最优的一对为 0.970）。**因此，按影响集划界能减少改写，而这一点只能由 R2 与 RA 的对比得出。**

降幅的大小与影响集排除了多少占用有关。影响集排除的占用越多，降幅越大；但被排除的占用本身只能解释 $0.604 - 0.540$ 这一差距的一部分，其余来自允许改写集合缩小后改路结果的变化。在影响集接近全部尚未结束占用的算例上（拥堵例，允许改写占比 0.94），降幅仅为 0.010；而在排除较多占用的算例上（mid 布局，0.91），降幅达 0.118。两个算例只能说明方向。为看逐对的分布，图 8 以允许改写占比 $\mathrm{Cl}/|R^{\circ}|$ 为横轴、RA 相对 R2 的改动比例降幅为纵轴，画出 50 对，并在本样本上取阈值 $\mathrm{Cl}/|R^{\circ}| = 0.93$：该线以上 23 对平均降幅 0.029，以下 27 对平均降幅 0.094。占比超过 0.97 的各对降幅都为零，其中有两种情况：占比为 1.00 的 6 对，影响集就是全部尚未结束的占用，R2 与 RA 的计算完全相同，降幅必然为零；其余各对的影响集排除了少量占用，降幅仍为零，这是实测结果。该阈值在本样本上选定，用于回答「何时按影响集划界仍有收益」，不作外推，也不同于影响集规模的结构预测（第 6.6 节）。

结论：在「不改写已经结束的占用」且「响应须在毫秒量级」两个约束下，综合四种方法的结果可以看到：R2 与 RA 耗时相同，说明毫秒量级的响应来自单遍改路、不重新启动搜索，而非来自预约影响集；两者完工时间相同，说明影响集不是一种优化手段；R2 的改动比例更低，因为影响集外的占用按命题 3 保持原计划，不被改写。**按影响集划界的收益体现在改写量上，而不在速度或完工时间上**：在耗时与完工时间都相同的条件下，被改写的占用占全表的比例，不按影响集划界（RA）为 0.604，按影响集划界（R2）为 0.540，相差约 6 个百分点，即 RA 比 R2 多改写约 12% 的占用。若允许秒级耗时，热启动全局重调度 R0+ 能得到更短的完工时间 $C_{\max}$（E5，第 6.7 节），但其耗时已超出毫秒量级。

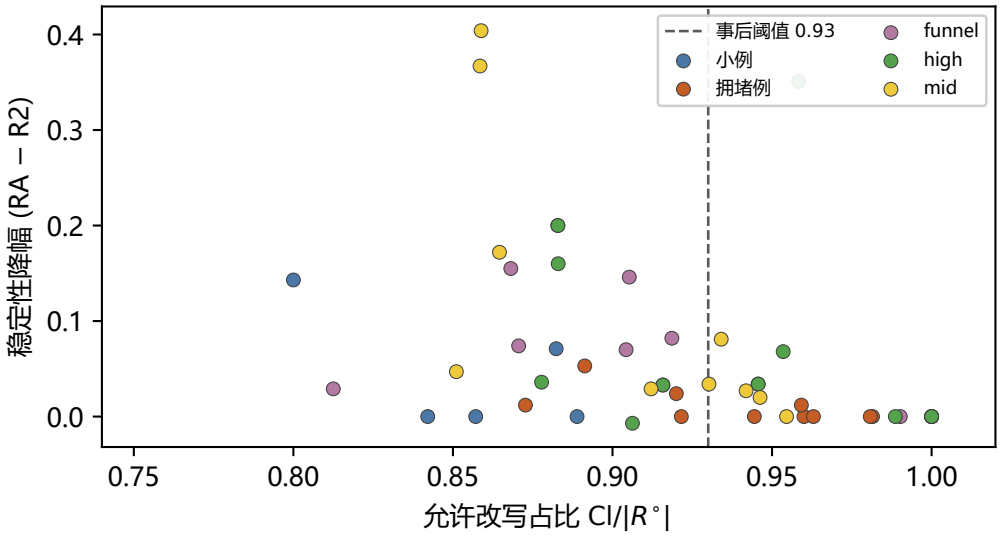


Fig. 8. E7 事后规则：允许改写占比与稳定性降幅（50 对，R2 对 RA）。虚线为样本内阈值 0.93；纵轴为正表示按影响集划界后改动比例更低。

表 10. 扰动规模扫描（拥堵例，10 个随机种子平均；有界修复启用范围扩展；R0+ 预算 2 s）。有界修复在各对上的耗时为 2.2–23.8 ms；R0+ $C_{\max}$ 的逐档四分位距为 8–76。

φ	有界修复可行	有界修复 ms	R0+ $C_{\max}$
0.10	100%	9.1	103.5
0.25	100%	11.1	107.0
0.50	100%	8.6	111.8
0.75	100%	8.4	125.8
1.00	100%	10.7	148.1

6.10 扰动规模扫描

本小节检验：扰动规模增大时，有界修复是否仍能恢复可行，耗时是否仍在毫秒量级。算例为拥堵例与小例，各 10 个随机种子；受扰工序比例 φ 取 0.10、0.25、0.50、0.75、1.00，$\varphi = 1$ 表示 t_{now} 之后的全部工序都受扰动。与 E3、E5 不同，本节的有界修复启用范围扩展；R0+ 预算为 2 s。

拥堵例的结果如表 10 与图 9 所示，从中可以看到三点。第一，五档 φ 下有界修复在每一对上都恢复可行，小例上同样全部可行。第二，有界修复的耗时没有随 φ 上升：拥堵例各档均值为 8.4–11.1 ms，各对为 2.2–23.8 ms，$\varphi = 1$ 时为 10.7 ms；小例约 1 ms。第三，R0+ 在每档的 $C_{\max}$ 都更优，与 E5 一致，但它用满 2 s 预算，耗时约高两个数量级。

结论：在所测扰动规模内，有界修复始终可行，耗时保持在十毫秒量级，不随扰动规模增长。

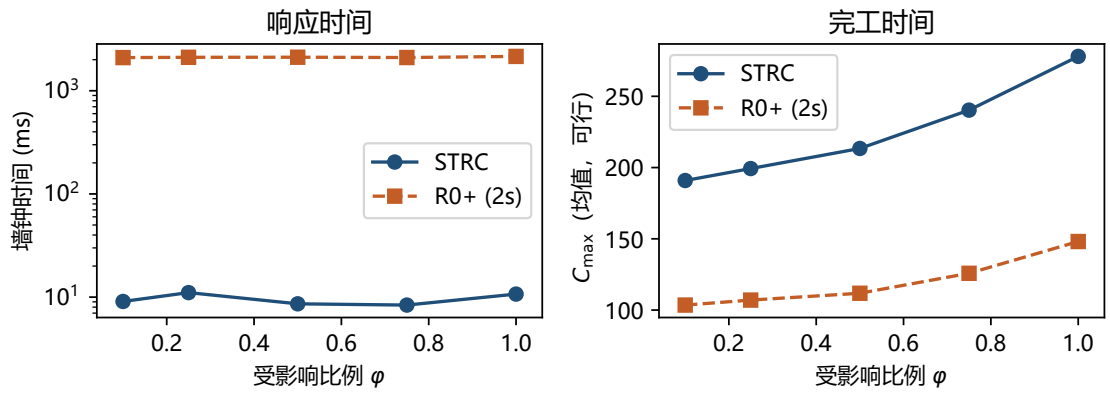


Fig. 9. 扰动规模扫描（拥堵例，10 个随机种子）：有界修复与 R0+ 的算法耗时（对数轴）与完工时间随 φ 变化。曲线取逐档均值，离散度口径与表 10 同（四分位距）。

表 11. E8 算例规模扫描（7 档 × 10 个随机种子）。「影响集」为预约影响集占全表之比，该比的可达上界是 $|R^{\circ}|/|R|$、本批均值约 0.61；换以可改写集合为分母则为 0.78–0.91（见正文）。耗时取中位数，「RD/R2」一列即两列中位数之比；正文讨论其单调性时用的是先在每对上求比值、再按档平均的结果，两种口径的档间起落一致但数值不同。末列为 R2 对 RS 的 $C_{\max}$ 胜负。

规模	$\lvert R\rvert$	影响集	R2 ms	RD ms	RD/R2	R2 对 RS
$8\times4\times4$	148	0.561	2.9	4.4	1.5×	7/1/2
$10\times5\times5$	172	0.485	3.2	6.1	1.9×	9/0/1
$12\times6\times6$	230	0.481	4.5	10.7	2.4×	7/1/2
$14\times7\times7$	281	0.492	6.1	14.5	2.4×	8/0/2
$16\times8\times8$	291	0.534	6.4	16.4	2.6×	10/0/0
$18\times9\times9$	356	0.535	8.2	26.4	3.2×	10/0/0
$20\times10\times10$	402	0.476	10.4	34.2	3.3×	10/0/0

6.11 E8：算例规模上的外推

上一节扫描的是扰动的规模，算例始终为 8 工件 4 机。本小节固定扰动设置、扩大算例本身，检验算例规模增大时，影响集占比是否随之接近 1、使「有界」失去实际意义，同时记录耗时的变化。

规模序列取工件数 $2k$、机器数 k、车辆数 k（$k=4,\dots,10$），即 $8\times4\times4$ 到 $20\times10\times10$，共 7 档 × 10 个随机种子 = 70 对。拥堵档、异构度、柔性度、运输/加工时长比与装卸站最小割、远端割全部按同一目标标定，只改变规模。三个对照方法与第 6.9 节定义相同。每档的算例均由同一随机种子 42 生成，只有一个拥堵档；E6 与扰动规模扫描未在这批算例上重新运行。

结果如表 11 所示，从中可以看到四点。

第一，七档 $\mathrm{Cl}/|R|$ 在 0.48–0.56 之间波动，且不单调（首档最高、末档最低，第五、六两档回到 0.53 以上），没有上升趋势。由于 Cl 只包含 t_{now} 之后尚未结束的占用，而本批尚未结束的占用只占全表约 0.61，$\mathrm{Cl}/|R|$ 最多只能到约 0.61，用它判断是否接近 1 没有意义。换用尚未结束的占用作分母后，七档 $\mathrm{Cl}/|R^{\circ}|$ 都在 0.78–0.91 之间（最小规模为 0.91，最大规模为 0.78，单对最高为 1.00），与 E6、E7 中的 92% 同量级。因此，影响集占比没有随算例规模增大而上升；但它仍覆盖尚未结束占用的 78%–91%，相对可改写的范围并不小。

第二，$|R|$ 增至 2.7 倍时，R2 的耗时中位数从 2.9 ms 增至 10.4 ms，即 3.6 倍，仍在十毫秒量级；七档数据不足以判断耗时随规模增长的快慢。首轮改路（不做范围扩展）即在 69/70 对上可行；唯一失败的一对（$10\times5\times5$，随机种子 42）经范围扩展在第 3 轮恢复可行，总耗时 12.0 ms，完工时间与首轮改路相同，故最终在 70/70 对上可行。

第三，耗时最短的全局重算 RD 与 R2 的耗时比从 1.5× 升至 3.3×。比值并不单调（先在每对上求比值再按档平均，$k=6\to7$ 处回落），但两端相差 1.8，远大于任一档内不超过 0.74 的四分位距。因此，规模越大，有界修复相对全局重算的耗时优势越明显。

第四，与全局右移相比，R2 的 $C_{\max}$ 在每档 10 对中胜 7–10 对（最后三档均为 10/0/0）；每档只有 10 对，看不出随规模的趋势。

结论：七档数据虽不足以推断更大规模下的趋势，但在 $8\times4\times4$ 至 $20\times10\times10$ 的范围内，影响集占比没有随算例规模上升，R2 耗时仍在十毫秒量级，且相对全局重算的耗时优势随规模扩大。

6.12 外部布局批次的复现

本小节检验 E1、E2、E3、E5 的结论是否依赖本文构造的布局。设置：换用 Lyu 等发表的布局 2–6（第 6.1 节第二批），每张布局 10 个随机种子，共 50 对，其余设置分别与 E1–E3、E5 相同；E5 只在布局 2 与布局 6 上运行，随机种子 $\{42,7,2024\}$、预算 $\{0.2,1,2\}$ s，共 18 个预算点。

结果与主批次逐项对应。E1：50/50 对上工序依赖图给出空范围，而原排程已不可行。E2：影响集对依赖边封闭（50/50），集外占用逐条不变（50/50）。E3：R1 可行 0/50，R2 可行 50/50。影响集占比：按 E1 的统计方式，各布局 $\mathrm{Cl}/|R|$ 的中位数为 0.438–0.547，主批次 50 对合并的中位数为 0.571；外部批次略低，两者都约为一半。E5：18 个预算点中 17 点 R0+ 的 $C_{\max}$ 更优，1 点持平（布局 6，随机种子 2024，预算 0.2 s），无一点更差；这些点已并入第 6.7 节的合并统计，交叉点仍不存在。

结论：换用已发表的布局后，E1、E2、E3、E5 的结论都不变，关于划界对象的结论（C1、C2）不依赖本文构造的布局。

6.13 小结

本章结果按其支撑的贡献归并；各项局限集中见第 7.1 节。

划界对象（C1、C2：按什么对象划界、划出的集合是否唯一确定）由四组结果支撑。走廊阻断下工序依赖图上的范围为空，而原排程确已不可行（E1，50 对上 50/50）；预约影响集对依赖边封闭，集外字段逐条不变（E2，50/50、50/50）；同一套改路程序只更换划界对象，恢复可行率即由 0/50 变为 50/50（E3）；在四类扰动与两种划界对象的交叉中 A/B 二分无例外，且 A 类上预约影响集均包含工序依赖图划界的结果（E6）。这些结果与命题 1–3 的结构陈述一一对应。因此，能否恢复可行取决于划界对象，而非改路程序。

划界的收益（C3：这样划界带来什么收益）。E7 中耗时最短的全局重算 RD 仍比有界修复慢（4.16 对 2.74 ms），说明毫秒量级的响应并非因为对照方法耗时过长，而是来自单遍改路。E5 中，热启动全局重调度在每个预算点上的 $C_{\max}$ 都不差于有界修复，放宽预算也不出现反超。换用不按影响集限制范围的 RA（与 R2 使用同一套改路，同样不改写已经结束的占用）后，耗时与 $C_{\max}$ 几乎不变，改动比例由 0.540 升至 0.604（逐对 31/18/1，$p=8.8\times10^{-7}$；第 6.9 节）。因此，按影响集划界的收益体现在改写更少，而不在速度或完工时间上。

其余结果限定了上述结论的适用范围：只由路网算出的静态指标无法预测影响集规模（E4）；受扰工序比例增至 1 时，有界修复仍全部可行，耗时在十毫秒量级（第 6.10 节）；算例规模扩大到 20 工件 10 机时，以尚未结束占用为分母的影响集占比没有上升（E8）；换用 Lyu 等发表的布局后，E1、E2、E3、E5 的结论均不变（第 6.12 节），说明关于划界对象的结论不依赖本文构造的布局。

7 总结

本文有三项贡献。**其一** (C1)，指出了一类工序依赖图无法刻画的失效：走廊阻断不改变任何工序指派，按受影响工序划出的范围为空，而原排程确已不可行（E1 中 50 对全部如此，第 6.2 节）。**其二** (C2)，以 STRC 得到的预约影响集作为重调度范围：依赖关系一经给定，一条占用是否属于该集合即唯一确定，集内没有指向集外的依赖边（E2 中 50 对全部满足，第 6.3 节）。**其三** (C3)，给出与该划界相匹配的局部恢复，并区分各项结果的来源：同一套改路程序只更换划界对象，恢复可行率即由 0/50 变为 50/50（第 6.4 节），因此可行与否取决于划界对象，而非改路程序的优劣。

让行关系、集内改路而集外保持原计划、失败后扩大允许改写的范围均沿用既有文献（第 2.5 节）。本文的主张是：工序依赖图按定义不含走廊占用之间的让行关系，因而无法刻画只使占用失效的扰动；在预约依赖图上划界后，须改写的占用由既定先后关系唯一确定，而非取决于启发式半径。

将允许改写的集合从预约影响集换为「t_{now} 之后尚未结束的全部占用」、其余条件不变时，算法耗时与完工时间几乎不变，改动比例却由 0.540 升至 0.604（第 6.9 节）；影响集比全部尚未结束的占用小得越多，这一差别越大，两者接近时差别趋于消失。因此，毫秒量级的响应来自单遍改路、不重新启动搜索；**按影响集划界的作用在于减少改写：集外占用保持原计划（命题 3）**。

7.1 局限

以下四条按对前文结果的影响排序，第 7.2 节逐条给出推进方向。

(1) **完工时间。**有界修复以完工时间换取算法耗时：在所测的 12 个算例–随机种子对上，热启动全局重调度的 $C_{\max}$ 都不差于有界修复，放宽预算也不出现反超（第 6.7 节）。该局限关系到划界的收益，不影响关于划界对象的结论（C1、C2）。

(2) **影响集的保证与规模。**命题 3 只给出集外占用不变的充分条件；依赖边集是否完备，没有经过独立的穷尽检验。影响集也不是恢复可行所需的最小集合：它覆盖尚未结束占用的约 92%，比把它们全部纳入改写只少不到一成，因此按影响集划界带来的改写减少有限，影响集越接近全部尚未结束的占用，这一收益越小。此外，只由路网算出的静态指标无法预测影响集的规模（第 6.6 节）。该局限关系到 C2 中保证的强度与影响集的大小，不影响 C1 的结论。

(3) **实现覆盖的问题范围。** B 类扰动只以改路响应；A 类另允许换车与改派，但不改变加工顺序；由假设 A3，每次扰动只做一次响应。放开加工顺序即向全局重调度退化，「有界」随之失去意义；多个扰动相继到达、扰动信息存在估计误差、随新信息反复重调度等情形均未涉及。算例规模已扩展至 20 工件 10 机，影响集占比未随规模上升，但 E8 只有七档，不足以推断更大规模下的趋势。因此结果向更大规模与连续扰动的外推仍受限。

(4) **外部参照。**这一类研究目前没有可换用的公开基准。公开算例族 [2, 3] 把运输时间发布为机器间行程时间矩阵而不是路网：两族共 11 张布局，逐张检验后没有一张能还原成与所发布矩阵一致的无向走廊图，因而占用表 R、预约依赖图 G_R、预约影响集 $\mathrm{Cl}(\cdot)$ 与 B 类扰动都无法在其上定义（第 6.1 节）。第二批算例的布局转录自 Lyu 等 [9] 的附录，但逐段行驶时间与工件数据仍由本文给定，不能用来复现 Lyu 或 van Os [23] 的参照值，其覆盖范围也小于主批次。因此本文算例中只有路网取自已发表文献，结果的外部效度有限；同一实现内各方法之间的相对比较不受影响。

7.2 后续工作

下列四条与上一节的四个局限一一对应。

一、缩小完工时间上的差距（局限 1）。主批次中，R0+ 与 RD 的解码都不改写已结束的占用（E5 的 18 个预算点与 E7 的 50 对上均为 0 次），因此两法完工时间的差距不来自改写已结束的占用。若要缩小差距，须在不改写已结束占用的前提下进行有限度的改派或改序，但这会使方法向全局重调度退化。

二、检验依赖边集的完备性，估计影响集与最小集合的差距，并预测其规模（局限 2）。*完备性*：在当前让行边集下，试探泄漏为 0；下一步是采用其他扰动形态（例如缩短占用，或延后超过 1 个时间单位）进行试探，而非继续补充同类依赖边。*与最小集合的差距*：在固定修复能力下定义最小允许改写集合，并给出影响集与之的差距；当前影响集覆盖约 92% 的尚未结束占用，两者的差距可能相当大。*规模刻画*：目前只有一条事后规则（允许改写占比，图 8），它随排程而变，无法在扰动前计算；而只由路网算出的静态指标在实际布局上取值恒定，同一张路网上只改变机器配置，影响集占比就会变化（第 6.6 节）。下一步可考察随排程而变的量，例如 t_{now} 附近受扰走廊上相互重叠的占用条数，由给定布局与当前排程预测影响集规模。

三、放开假设 A3（局限 3；A3 为单次、完全观测的假设）。下一步是放开该假设，考察连续扰动下影响集是否会逐次累积变大。

四、采用外部来源的行驶时间（局限 4）。布局已取自已发表文献，逐段行驶时间仍由本文给定。下一步需要一份同时发布路网与逐段行驶时间的算例族；E3 中的非均匀边权扫描只检验了结果对边权设定的稳健性，不能代替外部发布的行驶时间。

References

[1] Andrea D'Ariano, Dario Pacciarelli, and Marco Pranzo. 2007. A branch and bound algorithm for scheduling trains in a railway network. *European Journal of Operational Research* 183, 2 (2007), 643–657. doi:10.1016/j.ejor.2006.10.034

[2] Dalila B. M. M. Fontes, S. Mahdi Homayouni, and José F. Gonçalves. 2023. A hybrid particle swarm optimization and simulated annealing algorithm for the job shop scheduling problem with transport resources. *European Journal of Operational Research* 306, 3 (2023), 1140–1157. doi:10.1016/j.ejor.2022.09.006

[3] Seyed Mahdi Homayouni and Dalila B. M. M. Fontes. 2021. Production and Transport Scheduling in Flexible Job Shop Manufacturing Systems. *Journal of Global Optimization* 79, 2 (2021), 463–502. doi:10.1007/s10898-021-00992-6

[4] Ahmad Reza Jafarian-Moghaddam, Ali Esmailpoor Sanjabi, and Elham Mortazavi Moghaddam. 2026. Integrated tri-objective scheduling and internal logistics optimization in U-shaped flexible flow shops: a mathematical model and metaheuristic framework. *International Journal of Systems Science: Operations & Logistics* 13, 1 (2026), 2714000. doi:10.1080/23302674.2026.2714000

[5] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139

[6] Jürgen Kuster, Dietmar Jannach, and Gerhard Friedrich. 2007. Local Rescheduling: A novel approach for efficient response to schedule disruptions. In *2007 IEEE Symposium on Computational Intelligence in Scheduling (CISched)*. IEEE, Honolulu, HI, USA, 79–86. doi:10.1109/SCIS.2007.367673

[7] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543

[8] Sirui Li, Wenbin Ouyang, Yining Ma, and Cathy Wu. 2025. Learning-guided rolling horizon optimization for long-horizon flexible job-shop scheduling. arXiv:2502.15791 [cs.LG] doi:10.48550/arXiv.2502.15791

[9] Xiangfei Lyu, Yuchuan Song, Changzheng He, Qi Lei, and Weifei Guo. 2019. Approach to Integrated Scheduling Problems Considering Optimal Number of Automated Guided Vehicles and Conflict-Free Routing in Flexible Manufacturing Systems. *IEEE Access* 7 (2019), 74909–74924. doi:10.1109/ACCESS.2019.2919109

[10] Nir Malka, Guy Shani, and Roni Stern. 2024. Online Planning for Multi Agent Path Finding in Inaccurate Maps. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Abu Dhabi, United Arab Emirates, 10214–10221. doi:10.1109/IROS58592.2024.10801975

[11] François Margot, Maurice Queyranne, and Yaoguang Wang. 2003. Decompositions, network flows, and a precedence constrained single-machine scheduling problem. *Operations Research* 51, 6 (2003), 981–992. doi:10.1287/opre.51.6.981.24912

[12] Alessandro Mascis and Dario Pacciarelli. 2002. Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research* 143, 3 (2002), 498–517. doi:10.1016/S0377-2217(01)00338-1

[13] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355

[14] Djamila Ouelhadj and Sanja Petrovic. 2009. A Survey of Dynamic Scheduling in Manufacturing Systems. *Journal of Scheduling* 12, 4 (2009), 417–431. doi:10.1007/s10951-008-0090-8

[15] Fei Qiao, Li Li, Yumin Ma, and Bin Shi. 2012. Single Machine Oriented Match-Up Rescheduling Method for Semiconductor Manufacturing System. In *Intelligent Robotics and Applications (ICIRA 2012) (Lecture Notes in Computer Science, Vol. 7507)*. Springer, Berlin, Heidelberg, 217–226. doi:10.1007/978-3-642-33515-0_22

[16] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352

[17] Miguel A. Salido, Joan Escamilla, Federico Barber, and Adriana Giret. 2017. Rescheduling in job-shop problems for sustainable manufacturing systems. *Journal of Cleaner Production* 162 (2017), S121–S132. doi:10.1016/j.jclepro.2016.11.002

[18] Velusamy Subramaniam and Amritpal Singh Raheja. 2003. mAOR: A heuristic-based reactive repair mechanism for job shop schedules. *The International Journal of Advanced Manufacturing Technology* 22, 9-10 (2003), 669–680. doi:10.1007/s00170-003-1601-6

[19] Yong Jin Suh and Jin Young Choi. 2022. Efficient Fab Facility Layout with Spine Structure Using Genetic Algorithm under Various Material-Handling Considerations. *International Journal of Production Research* 60, 9 (2022), 2816–2829. doi:10.1080/00207543.2021.1904159

[20] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526

[21] Jiří Švancara, Marek Vlk, Roni Stern, Dor Atzmon, and Roman Barták. 2019. Online Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. AAAI Press, Honolulu, HI, USA, 7732–7739. doi:10.1609/aaai.v33i01.33017732

[22] Hongtao Tang, Xuewei Zhang, Jiawei Huang, Xuesong Xu, and Jiansha Lu. 2026. Integrated Scheduling of Multi-Objective Lot-Streaming Hybrid Flowshop with AGV under Dynamic Environments. *International Journal of Industrial Engineering Computations* 17, 2 (2026), 295–316. doi:10.5267/j.ijiec.2025.9.003

[23] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342

[24] Guilherme E Vieira, Jeffrey W Herrmann, and Edward Lin. 2003. Rescheduling Manufacturing Systems: A Framework of Strategies, Policies, and Methods. *Journal of Scheduling* 6, 1 (2003), 39–62. doi:10.1023/A:1022235519958

[25] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795

[26] Hongliang Zhang, Chaoqun Qin, Wenhui Zhang, Zhenxing Xu, Gongjie Xu, and Zhenhua Gao. 2023. Energy-Saving Scheduling for Flexible Job Shop Problem with AGV Transportation Considering Emergencies. *Systems* 11, 2 (2023), 103. doi:10.3390/systems11020103

[27] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, Kate Larson (Ed.). International Joint Conferences on Artificial Intelligence Organization, Jeju, South Korea, 311–320. doi:10.24963/ijcai.2024/35